

New Techniques for Random Probing Security and Application to Raccoon Signature Scheme

Sonia Belaïd, Matthieu Rivain, and Mélissa Rossi*

CryptoExperts, Paris, France
first.last@cryptoexperts.com

Abstract. The random probing model formalizes a leakage scenario where each wire in a circuit leaks with probability p . This model holds practical relevance due to its reduction to the noisy leakage model, which is widely regarded as the appropriate formalization for power and electromagnetic side-channel attacks.

In this paper, we present new techniques for designing efficient masking schemes that achieve tighter random probing security with lower complexity. First, we introduce the notion of *cardinal random probing composability* (Cardinal-RPC), offering a new trade-off between complexity and security for composing masking gadgets. Next, we propose a novel refresh technique based on a simple iterative process: randomly selecting and updating two shares with fresh randomness. While not perfectly secure in the standard probing model, this method achieves arbitrary cardinal-RPC security, making it a versatile tool for constructing random-probing secure circuits. Using this refresh, we develop additional basic gadgets (e.g., linear multiplication, addition, and copy) that satisfy the cardinal-RPC notion. Despite the increased complexity, the gains in security significantly outweigh the overhead, with the number of iterations offering useful flexibility.

To showcase our techniques, we apply them to lattice-based signatures. Specifically, we introduce a new random-probing composable gadget for sampling small noise, a key component in various post-quantum algorithms. To assess security in this context, we generalize the random probing security model to address auxiliary inputs and public outputs. We apply our findings to Raccoon, a masking-friendly signature scheme originally designed for standard probing security. We prove the secure composition of our new gadgets for key generation and signature computation, and show that our masking scheme achieves a superior security-performance tradeoff compared to previous approaches based on random probing expansion. To our knowledge, this is the first fully secure instantiation of a post-quantum algorithm in the random probing model.

Keywords: Post-quantum signature · Masking countermeasure · Random Probing Model · Raccoon Signature Scheme

1 Introduction

Most widely used cryptographic algorithms are considered secure against black-box attacks, where the adversary’s knowledge is restricted to certain inputs and outputs. However, as discovered in the late 1990s, their implementations on physical devices can be vulnerable to more sophisticated side-channel attacks. These attacks leverage physical emanations from the device, such as execution time, temperature, power consumption, or electromagnetic emissions during the algorithm’s execution. One of the most prevalent techniques to defend against side-channel attacks is masking, introduced independently in 1999 by Chari, Jutla, Rao, and Rohatgi [CJRR99], and by Goubin and Patarin [GP99]. In essence, the approach involves splitting each sensitive variable in an implementation into n shares, with $n - 1$ generated uniformly at random and the final share derived from the original value and the previous ones. Masking ensures that an adversary cannot recover the secret without all the shares.

* This work was done while the author was working at Agence Nationale de Sécurité des Systèmes d’Informations, Paris, France.

The probing model. To prove the security of masking schemes, it is essential to model the side-channel leakage an attacker could exploit, which leads to the concept of leakage models. One of the most well-known models is the d -probing model, introduced by Ishai, Sahai, and Wagner in [ISW03], which states that an attacker can gain access to the exact values of d intermediate variables, where d is a security parameter, but nothing more. The underlying intuition is that while all processed data may leak, combining noisy variables requires an exponential number of measurements as their number increases [CJRR99].

Most masking approaches consist in carefully composing small gadgets that are individually proven secure. This often necessitates using *refresh gadgets*, which functionally act as the identity function and help reduce share dependencies by introducing additional randomness. However, masking schemes secure in the d -probing model can become prohibitively expensive as the number of shares increases. While the complexity of linear operations scales linearly with the number of shares, the main bottleneck arises from non-linear operations, such as multiplying shared variables in $\mathbb{Z}_q$, which is common in post-quantum schemes. These operations significantly affect overall efficiency, complicating the balance between security and performance as the number of shares grows.

Masking-friendly post-quantum schemes. The use of masking in post-quantum NIST standards comes with a significant performance cost. While the slowdown remains manageable for the Crystals-Kyber key encapsulation mechanism [SAB⁺22], the impact on performance is much more severe for the Crystals-Dilithium signature scheme [LDK⁺22], as highlighted in [ABC⁺23,CGTZ23]. For other schemes, such as Falcon [PFH⁺22], designing efficient and provable masking in the d -probing model presents even greater challenges. To tackle the security-performance challenge, recent efforts have focused on designing masking-friendly schemes that reconsider the unmasked design to limit the use of non-linear operations. These schemes instead primarily rely on gadgets that can be efficiently masked and provably secure within the d -probing model. A first attempt to modify Falcon’s sampler in this purpose [EFG⁺22] was invalidated in [Pre23]. A more successful approach has emerged with the design of Raccoon [dEK⁺23], a masking-friendly analog to Crystals-Dilithium. While Raccoon produces larger signature sizes, it offers a significant speedup (by orders of magnitude for masked implementation). Similarly, a masking-friendly version of Falcon has been introduced in [EEN⁺24]. Both designs abandon the use of complex Gaussian distributions in favor of a simpler noise distribution based on small uniform additions.

The stronger random probing model. Although these schemes have made progress in reconciling security and efficiency, the d -probing model, on which they rely, has raised concerns about its practical relevance [BCPZ16]. The noisy leakage model, introduced by Prouff and Rivain [RP10] and inspired by [CJRR99], better reflects real-world devices by assuming all data leaks with noise. In 2014, Duc, Dziembowski, and Faust demonstrated that masking schemes that are secure under the conceptually-simpler *random probing model* are also secure against in the noisy leakage model [DDF14].

In this model, each wire leaks with a constant probability p , reflecting side-channel noise in practice. A circuit is secure in this model if the leakage can be simulated without knowledge of the secret, with a negligible failure probability, as originally formalized in [Ajt11]. This model enables provable security bounds against attackers, similar to traditional black-box security proofs. Furthermore, the random probing model not only addresses probing attacks but also captures horizontal attacks [BCPZ16], which exploit repeated manipulations of variables. Overall, this model provides significantly stronger theoretical security guarantees compared to the standard probing model. Several attempts have been made to design masking schemes secure in the random probing model [ISW03,Ajt11,ADF16,GJR18,AIS18,BCP⁺20]. However, most of these schemes are either impractical or fail to tolerate a constant leakage probability p . A notable breakthrough was made by Ananth, Ishai, and Sahai [AIS18], who introduced an expansion strategy built upon secure multi-party computation protocols. This approach was later refined by Belaïd et al. [BCP⁺20,BRT21,BRTV21], resulting in simpler *random probing expandable* gadgets that achieve arbitrary levels of random probing security for a fixed leakage probability p . Cassiers et al. further developed a tighter composition framework applied to the AES S-box in [CFOS21], though it struggles to scale to larger circuits and numbers of shares. In parallel, Berti, Faust, and Orlt [BFO23] introduced a generic compiler based on leakage diagrams, by employing a refresh gadget derived from Dziembowski et al. [DFZ19]. While promising, this approach requires specific

gadgets with carefully crafted composition properties to achieve efficient secure composition, making its generalization challenging. Jahandideh, Mennink, and Batina [JMB24] proposed an algebraic approach to evaluate random probing security, but it relies on estimations and assumes the adversary targets a single secret, limiting its applicability. As a result, extending these solutions to general secure circuit constructions remains non-trivial. All in all, no concrete instantiations exist for post-quantum schemes, which are expected to become widely adopted in the near future.

Our contributions. The primary goal of this paper is to make random probing security more practical for real-world implementations. We introduce new techniques which we apply to the use case of lattice-based signatures. Specifically, we focus our study on Raccoon [dEK⁺23], whose masking-friendly structure makes it an ideal starting point. Our work addresses all the key steps necessary to achieve this goal, ranging from extending security definitions to designing elementary gadgets and providing a comprehensive composition proof. More specifically, the contributions of this paper are as follows:

1. We first extend existing random-probing security notions, which already include various flavors (*e.g.* for composition and expansion), by adding two key features:
 - We extend the random-probing security framework to handle auxiliary inputs in the circuit. Such inputs are not masked and must hence tolerate some amount of leakage. In Raccoon’s masking proof, auxiliary inputs (a.k.a. “unshared inputs”) capture the small uniform randomness used for sampling noise. Additionally, we consider the possibility to expose public outputs, such as the public key in the key generation algorithm and the signature in the signing algorithm, as proposed in [BBE⁺18] in the context of standard probing security. The extended random-probing security notions are presented in Section 3.1.
 - We generalize the notion of random probing composability (RPC). Instead of a fixed threshold on the number of input and output shares, we consider all possible cardinalities of leaked output shares and required input shares for the simulation. This refined notion, termed *cardinal-RPC*, is detailed in Section 3.2. We also combine the concepts of auxiliary inputs/public outputs with cardinal-RPC in Section 3.3.
2. A crucial step in designing random probing secure gadgets is defining a refreshing gadget that randomizes the shares before performing the target operation. One of our key contributions is the design of a new random probing-friendly refresh gadget. The concept is simple: start with an encoding of zero and, in each iteration, add and subtract a random value to two randomly selected shares. After a certain number of iterations, the input to be refreshed is share-wise added to the result. As the number of iterations increase, the “quality” of the refreshing improves until a threshold is reached. In Section 4.1, we quantify this quality by introducing the new intermediary notion of *random probing partitioned* (RPP). We use this to prove that our new refresh is cardinal-RPC, with a tunable security advantage depending on the number of iterations. Taking a step forward, we integrate our new refresh design and composability definition into the necessary basic gadgets: addition, copy and linear multiplication (*i.e.* multiplication by a public value). We prove the cardinal-RPC security of these gadgets. These designs and proofs are detailed in Section 4.2.
3. In Section 5, we introduce the noise generation gadget essential for Raccoon. This design leverages new elementary gadgets to create an algorithm that combines small uniform random samples for generating Raccoon’s secrets. We employ two approaches for designing the secure gadget: (1) using *random probing expandable* (RPE) gadgets from [BCP⁺20, BRT21, BRTV21] and (2) using our new cardinal-RPC composition framework and gadgets. This section demonstrates a significant enhancement in complexity and randomness consumption with our composition proof and new gadgets.
4. Finally, in Section 6, we combine all concepts and gadgets to present a random-probing-secure instantiation of Raccoon, using parameters from the original scheme [dEK⁺23]. We evaluate both scenarios: (1) using expansion and (2) applying our general composition proof. Our detailed complexity analysis demonstrates that our new gadgets significantly enhance efficiency while achieving high levels of security. For example, for the Raccoon-128-16 instance with a leakage probability of 2^{-24} , we achieve 128-bit probing security with a randomness consumption that is between 10 and 20 times lower than with the

expansion strategy of [BCP⁺20] (the number of multiplications remaining similar in both approaches, while the number of additions scales with the randomness consumption).

All security and complexity results for our new proposal can be reproduced using the Python script available at <https://github.com/CryptoExperts/EC25-random-probing-Raccoon>.

2 Preliminaries

2.1 Notations

A probabilistic polynomial time algorithm (PPT) runs in time polynomial in the security parameter. Along the paper, $\mathbb{K}$ shall denote a finite field or a finite ring.¹ Sequences indexes are starting at 1. By convention, when the maximum index is 0, the sequence is empty, for example $x_1, \dots, x_0 = \emptyset$. For some tuple $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{K}^n$ and for some set $I \subseteq [n]$, the tuple $(x_i)_{i \in I}$ is denoted $\mathbf{x}|_I$. For an integer n , we denote $\mathcal{P}(n)$ the set of partitions of n , which is defined as:

$$\mathcal{P}(n) = \{(n_1, \dots, n_\ell) \mid \ell \in \mathbb{N} \wedge n_1 \geq n_2 \geq \dots \geq n_\ell \wedge n_1 + \dots + n_\ell = n\}.$$

Distributions. Two probability distributions, D_1 and D_2 , are ε -close, denoted as $D_1 \approx_\varepsilon D_2$, if their statistical distance is upper bounded by ε , namely

$$\text{SD}(D_1; D_2) := \frac{1}{2} \sum_x |p_{D_1}(x) - p_{D_2}(x)| \leq \varepsilon,$$

where $p_{D_1}(\cdot)$ and $p_{D_2}(\cdot)$ represent the probability mass functions of D_1 and D_2 , respectively. For two random variables X_1 and X_2 , we write $X_1 \stackrel{\text{id}}{=} X_2$ when $\varepsilon = 0$ i.e. when X_1 and X_2 are identically distributed.

Envelopes. In the proofs throughout this paper, exact probabilities are often challenging to compute directly, so we instead rely on upper bounds. The notion of probability envelope will be heavily used in this paper. Specifically, for a random variable X following a discrete distribution $\mathcal{D}$, we define $\mathcal{E}$ as an upper envelope of the distribution. That is, for any value x , the probability that X takes the value x (i.e. $\mathbb{P}(X = x)$) is bounded above by $\mathcal{E}(x)$, written as $X \lesssim \mathcal{E}$. As we will consider probability distributions conditioned to different disjoint events, we will often consider collections of probability envelopes with sometimes complex indices. For instance, we consider collections $\mathcal{E} = (\mathcal{E}_{(j_1, \dots, j_m)})$ indexed by tuples $(j_1, \dots, j_m) \in [0, n]^m$. In other words, each possible tuple value gives rise to an envelope associated with a distribution indexed by the same tuple.

Sharing. We use the general notation of $\llbracket x \rrbracket = (x_1, \dots, x_n) \in \mathbb{K}^n$ for a sharing of a secret value x .

2.2 Linear Sharing, Circuits, and Gadgets

In the following, the n -linear decoding mapping, denoted by LinDec , refers to the function $\mathbb{K}^n \rightarrow \mathbb{K}$ defined as

$$\text{LinDec} : (x_1, \dots, x_n) \mapsto x_1 + \dots + x_n,$$

for any $n \in \mathbb{N}$ and $(x_1, \dots, x_n) \in \mathbb{K}^n$. We shall further consider that, for every $n, \ell \in \mathbb{N}$, on input $(\llbracket x_1 \rrbracket, \dots, \llbracket x_\ell \rrbracket) \in (\mathbb{K}^n)^\ell$ the n -linear decoding mapping acts as

$$\text{LinDec} : (\llbracket x_1 \rrbracket, \dots, \llbracket x_\ell \rrbracket) \mapsto (\text{LinDec}(\llbracket x_1 \rrbracket), \dots, \text{LinDec}(\llbracket x_\ell \rrbracket)).$$

¹ Along the paper, $\mathbb{K}$ denotes the base structure (ring or field) of the arithmetic circuits. We use the notation $\mathbb{K}$ as those circuits are defined over a field most of the time. But in the context of our application to Raccoon, $\mathbb{K}$ shall be the ring $\mathbb{Z}_q$ with q a non-prime integer.

Definition 1 (Linear Sharing from [BCP⁺20]). Let $n, \ell \in \mathbb{N}$. For any $x \in \mathbb{K}$, an n -sharing of x is a random vector $\llbracket x \rrbracket \in \mathbb{K}^n$ such that $\text{LinDec}(\llbracket x \rrbracket) = x$. It is said to be uniform if for any set $I \subseteq [n]$ with $|I| < n$ the tuple $\llbracket x \rrbracket_I$ is uniformly distributed over $\mathbb{K}^{|I|}$. An n -linear encoding is a probabilistic algorithm LinEnc which on input a tuple $\mathbf{x} = (x_1, \dots, x_\ell) \in \mathbb{K}^\ell$ outputs a tuple $\llbracket \mathbf{x} \rrbracket = (\llbracket x_1 \rrbracket, \dots, \llbracket x_\ell \rrbracket) \in (\mathbb{K}^n)^\ell$ such that $\llbracket x_i \rrbracket$ is a uniform n -sharing of x_i for every $i \in [\ell]$.

An *arithmetic circuit* on a finite field (or finite ring) $\mathbb{K}$ is a labeled directed acyclic graph whose edges are wires and vertices are *arithmetic gates* processing operations on $\mathbb{K}$. The circuit is built from a base set of gates $\mathbb{B} = \{g : \mathbb{K}^\ell \rightarrow \mathbb{K}^m\}$, such as addition gates $(x_1, x_2) \mapsto x_1 + x_2$, multiplication gates $(x_1, x_2) \mapsto x_1 \cdot x_2$, and copy gates $x \mapsto (x, x)$. A *randomized arithmetic circuit* includes a random gate that outputs a fresh uniform random value from $\mathbb{K}$. For a randomized circuit C , when we write $C(\rho, \cdot)$, this refers to the corresponding de-randomized circuit with the random tape $\rho \in \{0, 1\}^*$ as input.

In the following, we shall call an $(n\text{-share}, \ell\text{-to-}m)$ *gadget*, a randomized arithmetic circuit that maps an input $\llbracket \mathbf{x} \rrbracket \in (\mathbb{K}^n)^\ell$ to an output $\llbracket \mathbf{y} \rrbracket \in (\mathbb{K}^n)^m$ such that $\mathbf{x} = \text{LinDec}(\llbracket \mathbf{x} \rrbracket) \in \mathbb{K}^\ell$ and $\mathbf{y} = \text{LinDec}(\llbracket \mathbf{y} \rrbracket) \in \mathbb{K}^m$ satisfy $\mathbf{y} = g(\mathbf{x})$ for some function g . We recall the definition of a circuit compiler in Appendix A (introduced in [BCP⁺20]).

2.3 Random Probing Security

Let $p \in [0, 1]$ be some constant leakage probability parameter, a.k.a. the *leakage rate*. In the p -random probing model, an evaluation of a circuit C leaks the value carried by each wire with a probability p , all the wire leakage events being mutually independent.

As in [BCP⁺20], we formally define the random-probing leakage of a circuit from the two following algorithms:

- The *leaking-wires sampler* takes as input a randomized arithmetic circuit C and a probability $p \in [0, 1]$, and outputs a set $\mathcal{W}$, denoted as

$$\mathcal{W} \leftarrow \text{LeakingWires}(C, p) ,$$

where $\mathcal{W}$ is drawn by including each wire label from the circuit C with probability p to $\mathcal{W}$ (where all the probabilities are mutually independent).

- The *assign-wires sampler* takes as input a deterministic arithmetic circuit C , a fixed random tape $\rho \in \{0, 1\}^*$, a set of wire labels $\mathcal{W}$ (subset of all the wire labels of C), and a shared input $\llbracket x \rrbracket$, and it outputs a $|\mathcal{W}|$ -tuple $\mathbf{w} \in \mathbb{K}^{|\mathcal{W}|}$, denoted as

$$\mathbf{w} \leftarrow \text{AssignWires}(C, \rho, \mathcal{W}, \llbracket x \rrbracket) ,$$

where $\mathbf{w}$ corresponds to the exact assignments of the wires with label in $\mathcal{W}$ for an evaluation of $C(\rho, \llbracket x \rrbracket)$.

Note that $\mathbf{w}$ is deterministically fixed from $C, \rho, \mathcal{W}$ and $\llbracket x \rrbracket$. Let us now induce a distribution for $\mathbf{w}$ as a random probing leakage with fixed leaked wires.

Definition 2 (Random Probing Leakage for Fixed Leaked Wires). Let C be a deterministic arithmetic circuit, p be a probability, $\mathcal{W}$ be a set of wire labels and $\llbracket x \rrbracket$ be a shared input of C . Let $|\rho|$ be the size of the random tape for C and R be a uniform random variable on $\{0, 1\}^{|\rho|}$. We define the leakage for $\mathcal{W}$ as the following random variable

$$\mathcal{L}_{\mathcal{W}}(C, \llbracket x \rrbracket) := \text{AssignWires}(C, R, \mathcal{W}, \llbracket x \rrbracket) .$$

Following [BCRT23], we shall say that a pair of vectors $(\llbracket x \rrbracket, \llbracket y \rrbracket) \in (\mathbb{K}^n)^2$ is *admissible* for a randomized 1-to-1 circuit C if there exists a random tape $\rho_{\llbracket x \rrbracket, \llbracket y \rrbracket}$ such that $\llbracket y \rrbracket = C(\rho_{\llbracket x \rrbracket, \llbracket y \rrbracket}, \llbracket x \rrbracket)$.

Definition 3 (Induced Random Probing Leakage for Fixed Leaked Wires). Let C be a deterministic arithmetic circuit, p be a probability and $\mathcal{W}$ be a set of wire labels. Let $(\llbracket x \rrbracket, \llbracket y \rrbracket)$ be an admissible pair of vectors for C and $R_{\llbracket x \rrbracket, \llbracket y \rrbracket}$ be a uniform random variable on the set of random tapes ρ such that $\llbracket y \rrbracket = C(\rho, \llbracket x \rrbracket)$. We define the leakage for $\mathcal{W}$ as the following random variable

$$\mathcal{L}_{\mathcal{W}}(C, \llbracket x \rrbracket, \llbracket y \rrbracket) := \text{AssignWires}(C, R_{\llbracket x \rrbracket, \llbracket y \rrbracket}, \mathcal{W}, \llbracket x \rrbracket) .$$

Let us now introduce the formal definition of the so-called random probing security (RPS for short).

Definition 4 (Random Probing Security). *A randomized arithmetic circuit C is (p, ϵ) -random probing secure with respect to encoding Enc if there exists a PPT simulator² Sim such that for every input x , the distribution of $\text{Sim}(C)$ is ϵ -close to the distribution of $\mathcal{L}_{\mathcal{W}}(C, \llbracket x \rrbracket)$ where*

$$\begin{aligned} \mathcal{W} &\leftarrow \text{LeakingWires}(C, p) && \text{Drawing the leaking wires.} \\ \llbracket x \rrbracket &\leftarrow \text{Enc}(x). && \text{Drawing an encoding of the input.} \end{aligned}$$

Informally, while the random probing allows to consider that a large amount of wires can leak at the same time, the leakage itself is balanced by the probability of such an event. For example, if $\mathcal{W}$ corresponds to the labels of all the shares of the input x , the simulator will not be able to draw a correct sample without knowing x . However, this event happens with a probability p^n (n being the number of shares) which becomes negligible as n grows.

3 Extensions of the Random Probing Security

In this section we introduce the notion of *random probing security with auxiliary inputs and public outputs* (RPS-AI-O) as well as associated notions for the secure composition of gadgets. We also introduce the notion of *cardinal random probing composability* (cardinal-RPC) which provides a new trade-off between complexity and tightness for the composition compared to existing notions. We finally provide general composition results in this framework.

3.1 Random Probing Security with Auxiliary Inputs and Public Outputs

The notion of random probing security with auxiliary inputs and public outputs is a generalization of the RPS notion (Definition 4). It considers a randomized arithmetic circuit with shared inputs and outputs (as in the standard RPS notion) with two extra powers for the simulator.

1. First, the circuit admits additional auxiliary inputs which are not in shared form and for which partial information is potentially leaked. This partial information is captured by letting the simulator requesting a set of coordinates for each auxiliary input. The cardinality of these sets follow a list of distributions which is a parameter of the achieved notion.
2. Secondly, additional (unmasked) outputs are fully given to the simulator.

We introduce these new features in order to relax the RPS and achieve better complexity-security trade-offs for cryptosystems using random nonces that can be partly leaked without compromising the security. A typical example is the masked lattice-based signature scheme Raccoon. In this signature, some random nonces can be exposed to a probing attacker without harming the security. In security proofs, these random nonces are treated not as shares but as additional inputs [dKPR24]. This is formalized in the following definition.

² In concrete instantiations, the simulator Sim starts with generating a set of leaking wires through, as explicated in other definitions. Specifically, Sim will be decomposed with another PTT algorithm Sim^* as follows

$$\begin{aligned} \text{Sim}(C) : \quad & \mathcal{W} \leftarrow \text{LeakingWires}(C, p) \\ & \text{out} \leftarrow \text{Sim}^*(C, \mathcal{W}). \end{aligned}$$

Experiment 1: RPS-AI-O

$(I_1, \dots, I_k) \leftarrow \text{Sim}_1(C)$

The simulator chooses indexes for the auxiliary inputs.

$\text{out} \leftarrow \text{Sim}_2(\mathbf{a}_1|_{I_1}, \dots, \mathbf{a}_k|_{I_k}, \mathbf{z})$

The simulator is given the values of the auxiliary inputs. Next, it returns simulated values for the leaking wires.

return $(I_1, \dots, I_k, \text{out})$

Definition 5 (Random Probing Security with Auxiliary Inputs and Public Outputs). Let $n, \ell, m, k, d, \alpha \in \mathbb{N}$. Let C be a randomized arithmetic circuit with the following input/output partition³

$$C : \overbrace{(\mathbb{K}^n)^\ell}^{\text{masked inputs}} \times \overbrace{(\mathbb{K}^\alpha)^k}^{\text{auxiliary inputs}} \rightarrow \overbrace{(\mathbb{K}^n)^m}^{\text{masked outputs}} \times \overbrace{\mathbb{K}^d}^{\text{public outputs}}$$

$$(\llbracket \mathbf{x} \rrbracket, \mathbf{a}_1, \dots, \mathbf{a}_k) \mapsto (\llbracket \mathbf{y} \rrbracket, \mathbf{z}).$$

Let $\mathcal{E} = \mathcal{E}_1, \dots, \mathcal{E}_k$ be a set of probability distribution envelopes over the discrete set $[0, \alpha]$.

The circuit C is $(p, \epsilon, \mathcal{E})$ -random probing secure with auxiliary inputs and public outputs $((p, \epsilon, \mathcal{E})$ -RPS-AI-O) with respect to encoding Enc if there exists a PPT stateful two-stage simulator $(\text{Sim}_1, \text{Sim}_2)$ such that, for every admissible pair $((\llbracket \mathbf{x} \rrbracket, \mathbf{a}_1, \dots, \mathbf{a}_k), (\llbracket \mathbf{y} \rrbracket, \mathbf{z}))$, the outputs of Experiment 1 are such that

1. $|I_1| \lesssim \mathcal{E}_1, \dots, |I_k| \lesssim \mathcal{E}_k$, and
2. $\text{out} \approx_\epsilon \underbrace{\mathcal{L}_W(C, (\llbracket \mathbf{x} \rrbracket, \mathbf{a}_1, \dots, \mathbf{a}_k), (\llbracket \mathbf{y} \rrbracket, \mathbf{z}))}_{\text{Induced random probing leakage}}$ where $W \leftarrow \text{LeakingWires}(C, p)$ and $\llbracket \mathbf{x} \rrbracket \leftarrow \text{Enc}(\mathbf{x})$.

3.2 Cardinal Random Probing Composability

The notion of *random probing composability* (RPC) was initially introduced in [BCP⁺20]. In essence, a circuit composed of individual gadgets that satisfy the RPC property will itself be RPC, and, as a result, random probing secure. Specifically, a gadget is (t, p, ϵ) -RPC with some integer $t < n$ and $p, \epsilon \in [0, 1]$ if, given t output shares, the probability that more than t shares of each input are required to simulate the leakage of the gadget – where each wire leaks with probability p – along with the t output shares, is upper bounded by ϵ . Based on this property, a circuit C composed of (t, p, ϵ_i) -RPC gadgets $(G_i)_{1 \leq i \leq |C|}$ will itself be (t, p, ϵ) -RPC with $\epsilon = 1 - \prod_{1 \leq i \leq |C|} (1 - \epsilon_i)$. The formal RPC definition and the latter composition theorem are formally recalled in Appendix A.

In this paper, we shall refer to the original RPC notion as *threshold-RPC* because it is defined with respect to a threshold t on the tolerated leakage on the input and output sharings. An alternative notion, based on so-called *probe distribution tables* (PDT) and targeting tighter composition, was put forward in [CFOS21]. Instead of fixing a threshold t for both the number of output shares (to be simulated) and the number of input shares (required for the simulations), the PDT of a gadget considers each specific pair of sets of input shares and output shares. Specifically, each cell (I, J) in the table represents the probability that a particular set of input shares I (represented by the row) is required to simulate the leakage of a gadget – with each wire leaking with probability p – along with a corresponding set of output shares J (represented by the column). Based on such PDT for each gadget, the authors of [CFOS21] define several composition rules to upper bound the random probing security of the global circuit.

Similarly, we shall refer to the PDT notion as *general-RPC* because it is the most general RPC notion in the sense that it encompasses the original (threshold-)RPC notion as well as the cardinal-RPC notion that

³ To be more generic, one can consider different dimensions for the auxiliary inputs i.e. $\mathbb{K}^{\alpha_1}, \dots, \mathbb{K}^{\alpha_k}$ instead of $(\mathbb{K}^\alpha)^k$. The definition can be straightforwardly adapted.

Experiment 2: Cardinal-RPC

$\mathcal{W} \leftarrow \text{LeakingWires}(G, p)$ $I_1, \dots, I_\ell \leftarrow \text{Sim}_1(G, \mathcal{W}, J_1, \dots, J_m)$ $out_1, out_2 \leftarrow \text{Sim}_2(\llbracket x_1 \rrbracket_{I_1}, \dots, \llbracket x_\ell \rrbracket_{I_\ell})$ $\text{return } (I_1, \dots, I_\ell, out_1, out_2)$	<i>Drawing the leaking wires.</i> <i>The simulator is given the indexes of the leaking outputs. It chooses indexes for the inputs.</i> <i>The simulator is given the shares at desired indexes and returns simulated values for the leaking wires (out_1) and outputs (out_2)</i>
---	---

we propose here. The definition of general-RPC with our formalism is given in Appendix A. This notion provides tighter composition than the threshold-RPC notion, which can be viewed as a special case of the former. Specifically, the t -threshold-RPC security parameter ϵ of a gadget can be easily recalculated from its PDT. For each column with an output set of cardinality at most t , we compute the sum of the rows corresponding to input sets with cardinality strictly greater than t . The largest of these sums across all eligible columns determines the threshold-RPC security parameter ϵ .

The cardinal-RPC notion provides a practical trade-off between the threshold RPC notion and the general RPC notion. In essence, rather than using a fixed threshold t and for the number of shares required for each output and input, the cardinal-RPC notion considers the possible cardinalities for each of them. This makes it simpler than the general RPC notion, which considers all possible sets of shares for each input and output – a number that grows exponentially with the number of shares. On the other hand, the combinations of cardinalities only grow polynomially with the number of shares.

In the following definition, we use a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, n]^\ell$. It means that there are $(n+1)^m$ functions $\mathcal{E}_{j_1, \dots, j_m}$ such that

$$\forall j_1, \dots, j_m, i_1, \dots, i_\ell \in [0, n]^{m+\ell}, \quad \mathcal{E}_{j_1, \dots, j_m}(i_1, \dots, i_\ell) \in [0, 1] .$$

Definition 6 (Cardinal Random Probing Composability). *Let $n, \ell, m \in \mathbb{N}$. Let $\mathcal{E}$ represent a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, n]^\ell$. An n -share gadget $G : (\mathbb{K}^n)^\ell \rightarrow (\mathbb{K}^n)^m$ is $(p, \mathcal{E})$ -cardinal random probing composable ($(p, \mathcal{E})$ -cardinal-RPC) for some $p \in [0, 1]$ if there exists a PPT stateful two-stage simulator $(\text{Sim}_1, \text{Sim}_2)$ such that for every shared input $\llbracket \mathbf{x} \rrbracket \in (\mathbb{K}^n)^\ell$ and for every set collection $(J_1, \dots, J_m)$ where $J_1 \subseteq [n], \dots, J_m \subseteq [n]$, the outputs of Experiment 2 are such that*

1. $(|I_1|, \dots, |I_\ell|) \lesssim \mathcal{E}_{|J_1|, \dots, |J_m|}$, and
- 2.

$$(out_1, out_2) \stackrel{id}{=} \left(\underbrace{\mathcal{L}_{\mathcal{W}}(G, \llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{y} \rrbracket)}_{\text{Induced Random probing leakage}}, \underbrace{(\llbracket y_1 \rrbracket_{J_1}, \dots, \llbracket y_m \rrbracket_{J_m})}_{\text{Output leakage}} \right),$$

where $\mathcal{W} \leftarrow \text{LeakingWires}(G, p)$ and $\llbracket \mathbf{y} \rrbracket \leftarrow G(\llbracket \mathbf{x} \rrbracket)$.

We emphasize the broadness of Definition 6: every gadget can be cardinal RPC for some probability and envelope. Hence, it only makes sense to claim that a gadget is $(p, \mathcal{E})$ -cardinal-RPC for some explicit p and $\mathcal{E}$.

The security advantage for the threshold-RPC property can be derived from the distribution envelope of the cardinal-RPC property in a similar way that it is derived from the PDT of the general RPC notion as explained above. This implication is formally stated in Lemma 1 hereafter.

Although cardinal-RPC provides a looser bound compared to general-RPC, the two notions become equivalent in the case of a uniformly ℓ -to- m cardinal-RPC gadget. A gadget is uniformly cardinal-RPC when, for any $(t_1, \dots, t_\ell) \in [0, n]^\ell$, all input shares of cardinality $(t_1, \dots, t_\ell)$ – necessary for simulating

$(\llbracket \mathbf{y} \rrbracket, \mathbf{z}) \leftarrow G(\llbracket \mathbf{x} \rrbracket, \mathbf{a})$ $\mathcal{W} \leftarrow \text{LeakingWires}(C, p)$ $(I_1, \dots, I_\ell), (L_1, \dots, L_k) \leftarrow \text{Sim}_1(G, \mathcal{W}, \mathbf{z}, J_1, \dots, J_m)$	<i>Drawing the public outputs.</i> <i>Drawing the leaking wires.</i> <i>The simulator is given the indexes of the leaking outputs. It chooses indexes for the shared and auxiliary inputs.</i>
$out_1, out_2 \leftarrow \text{Sim}_2(\llbracket x_1 \rrbracket _{I_1}, \dots, \llbracket x_\ell \rrbracket _{I_\ell}, \mathbf{a}_1 _{L_1}, \dots, \mathbf{a}_k _{L_k})$	<i>The simulator is given the shares at desired indexes and returns simulated values for the leaking wires and outputs.</i>
$\text{return } (out_1, out_2, (I_1, \dots, I_\ell), (L_1, \dots, L_k))$	

both the leakage and the specified output – occur with equal probability. Furthermore, this equal probability distribution also applies to outputs of the same cardinality, ensuring symmetry in the simulation. The formal definition is given in Appendix A.

3.3 Cardinal-RPC with Auxiliary Inputs and Public Outputs

We now extend the previously introduced notion of cardinal-RPC to gadgets with auxiliary inputs and public outputs. The following definition is naturally obtained from cardinal-RPC (Definition 6) and RPS-AI-O (Definition 5).

Definition 7 (Cardinal RPC with Auxiliary Inputs and Public Outputs).

Let $n, \ell, m, k, \alpha, d \in \mathbb{N}$. Let $\mathcal{E}$ represent a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, n]^\ell$. Let $\mathcal{E}'$ represent a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, \alpha]^k$. Let G be a gadget with the following input/output partition

$$G : \overbrace{(\mathbb{K}^n)^\ell}^{\text{masked inputs}} \times \overbrace{(\mathbb{K}^\alpha)^k}^{\text{auxiliary inputs}} \rightarrow \overbrace{(\mathbb{K}^n)^m}^{\text{masked outputs}} \times \overbrace{\mathbb{K}^d}^{\text{public outputs}}$$

$$(\llbracket \mathbf{x} \rrbracket, \mathbf{a}_1, \dots, \mathbf{a}_k) \mapsto (\llbracket \mathbf{y} \rrbracket, \mathbf{z}).$$

The gadget G is $(p, \mathcal{E}, \mathcal{E}')$ -cardinal random probing composable with auxiliary inputs and public outputs $((p, \mathcal{E}, \mathcal{E}')$ -cardinal-RPC-AI-O) for some $p \in [0, 1]$ if there exists a PPT stateful two-stage simulator $(\text{Sim}_1, \text{Sim}_2)$ such that for every shared input $\llbracket \mathbf{x} \rrbracket \in (\mathbb{K}^n)^\ell$, auxiliary input $\mathbf{a}_1, \dots, \mathbf{a}_k \in (\mathbb{K}^\alpha)^k$, and for every set collection $(J_1, \dots, J_m)$ where $J_1 \subseteq [n], \dots, J_m \subseteq [n]$, the outputs of Experiment 3 are such that

1. $(|I_1|, \dots, |I_\ell|) \lesssim \mathcal{E}_{|J_1|, \dots, |J_m|}$,
2. $(|L_1|, \dots, |L_k|) \lesssim \mathcal{E}'_{|J_1|, \dots, |J_m|}$, and
- 3.

$$(out_1, out_2) \stackrel{id}{=} \left(\underbrace{\mathcal{L}_{\mathcal{W}}(G, (\llbracket \mathbf{x} \rrbracket, \mathbf{a}), (\llbracket \mathbf{y} \rrbracket, \mathbf{z}))}_{\text{Induced Random probing leakage}}, \underbrace{(\llbracket y_1 \rrbracket|_{J_1}, \dots, \llbracket y_m \rrbracket|_{J_m})}_{\text{Output leakage}} \right)$$

where $\mathcal{W} \leftarrow \text{LeakingWires}(C, p)$ and $(\llbracket \mathbf{y} \rrbracket, \mathbf{z}) \leftarrow G(\llbracket \mathbf{x} \rrbracket, \mathbf{a})$.

Similarly, one can define threshold-RPC-AI-O from the original threshold-RPC definition. See Definition 17 in appendix for details.

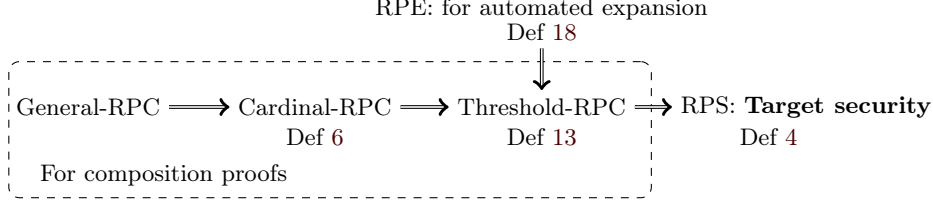


Fig. 1: Schematic implications between security notions

3.4 Summary of the notions

There exist many implications between the presented security notions. The main ones are shown in the schematic graph of Figure 1.

Lemma 1 (Cardinal-RPC $\implies$ Threshold-RPC). *Let $n, \ell, m \in \mathbb{N}$. Let $\mathcal{E}$ represent a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, n]^\ell$. If an n -share gadget $G : (\mathbb{K}^n)^\ell \rightarrow (\mathbb{K}^n)^m$ is $(p, \mathcal{E})$ -cardinal-RPC for some $p \in [0, 1]$, then it is (t, p, ϵ) -threshold-RPC for the same probability p and for any integer $t \in [0, n - 1]$, with*

$$\epsilon \leq \max_{(J_1, \dots, J_m) \in \mathbf{J}(t)} \left(\sum_{(I_1, I_2, \dots, I_\ell) \in \mathbf{I}(t)} \mathcal{E}_{|J_1|, \dots, |J_m|}(|I_1|, \dots, |I_\ell|) \right)$$

with $\mathbf{I}(t) = [(I_1, I_2, \dots, I_\ell) \in [0, n]^\ell \text{ such that } \exists i \in [1, \ell], |I_i| > t]$,

and $\mathbf{J}(t) = [(J_1, J_2, \dots, J_m) \in [0, n]^m \text{ such that } |J_1| \leq t, |J_2| \leq t, \dots, |J_m| \leq t]$.

Naturally, an n -share cardinal-RPC gadget is also cardinal-RPC-AI-O with auxiliary inputs of cardinality that is a multiple of n . This implication is established in Lemma 2 and follows directly from the definitions and the application of the law of total probability.

Lemma 2 (Cardinal-RPC-O $\implies$ Cardinal-RPC-AI-O). *Let $n, \ell, m, k, d \in \mathbb{N}$. Let $\mathcal{E}$ represent a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, n]^{\ell+k}$. If a gadget $G : (\mathbb{K}^n)^{\ell+k} \rightarrow (\mathbb{K}^n)^m \times \mathbb{K}^d$ is $(p, \mathcal{E})$ -cardinal-RPC-O, then it is also $(p, \mathcal{E}^x, \mathcal{E}^a)$ -cardinal-RPC-AI-O where $\mathcal{E}^x$ and $\mathcal{E}^a$ are defined, for all $j_1, \dots, j_m$, for all $i_1, \dots, i_\ell \in [0, n]^\ell$, for all $i'_1, \dots, i'_k \in [0, n]^k$ as*

$$\mathcal{E}_{j_1, \dots, j_m}^x(i_1, \dots, i_\ell) = \sum_{(t_1, \dots, t_k) \in [0, n]^k} \mathcal{E}_{j_1, \dots, j_m}(i_1, \dots, i_\ell, t_1, \dots, t_k),$$

$$\mathcal{E}_{j_1, \dots, j_m}^a(i'_1, \dots, i'_k) = \sum_{(t_1, \dots, t_\ell) \in [0, n]^\ell} \mathcal{E}_{j_1, \dots, j_m}(t_1, \dots, t_\ell, i'_1, \dots, i'_k).$$

While the cardinal-RPC-AI-O property is introduced for compositional purposes, it is important to highlight that a cardinal-RPC-AI-O circuit is also threshold-RPC-AI-O (following similar arguments to those of Lemma 1) and RPS-AI-O, as formalized in Lemma 3.

Lemma 3 (Cardinal-RPC-AI-O $\implies$ RPS-AI-O). *Following Definition 7, let $n, \ell, m, k, \alpha \in \mathbb{N}$. Let G be a gadget with the following input/output partition $(\mathbb{K}^n)^\ell \times (\mathbb{K}^\alpha)^k \rightarrow (\mathbb{K}^n)^m \times \mathbb{K}^d$. Let $\mathcal{E}$ be a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, n]^\ell$. Let $\mathcal{E}'$ be a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, \alpha]^k$. If the gadget G is $(p, \mathcal{E}, \mathcal{E}')$ -cardinal-RPC-AI-O, then it is also $(p, \epsilon, \mathcal{E}^*)$ -RPS-AI with*

$$\epsilon \geq 1 - \sum_{(n_1, n_2, \dots, n_\ell) \in [0, n-1]^\ell} \mathcal{E}_{(0, \dots, 0)}(n_1, n_2, \dots, n_\ell) \quad \text{and} \quad (\mathcal{E}_1^*, \dots, \mathcal{E}_k^*) = \mathcal{E}'_{(0, \dots, 0)}.$$

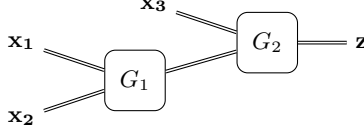


Fig. 2: Composition between $G_1 : (\mathbb{K}^n)^2 \rightarrow \mathbb{K}^n$ and $G_2 : (\mathbb{K}^n)^2 \rightarrow \mathbb{K}^n$.

Let us define $(\text{Sim}_1^*, \text{Sim}_2^*)$ as follows.

$\text{Sim}_1^*(G, \mathcal{W}, J)$: Separate $\mathcal{W}$ in $\mathcal{W}_1, \mathcal{W}_2$ for leaking wires in G_1 and G_2
 $(I_1^2, I_2^2) \leftarrow \text{Sim}_1^2(C, \mathcal{W}_2, J)$
 $(I_1^1, I_2^1) \leftarrow \text{Sim}_1^1(C, \mathcal{W}_1, I_2^2)$
return (I_1^1, I_2^1, I_2^2) .

$\text{Sim}_2^*(\llbracket x_1 \rrbracket|_{I_1}, \llbracket x_2 \rrbracket|_{I_2}, \llbracket x_3 \rrbracket|_{I_3})$: $out_1^1, out_2^1 \leftarrow \text{Sim}_2^1(\llbracket x_1 \rrbracket|_{I_1}, \llbracket x_2 \rrbracket|_{I_2})$
 $out_1^2, out_2^2 \leftarrow \text{Sim}_2^2(\llbracket x_3 \rrbracket|_{I_3}, out_2^1)$
return $(out_1^1, out_2^1), out_2^2$

Let us compute the distribution envelope. We denote by $I_{i,j} = |I_j^i|$ and $O = |J|$ where I_j^i and J are defined in Sim_1^* . By definition, $\forall \mathbf{t}_{\text{in}} = (t_{\text{in}}^1, t_{\text{in}}^2, t_{\text{in}}^3) \in [0, n]^3$, $\mathcal{E}_{t_{\text{out}}}^3(\mathbf{t}_{\text{in}})$ is an upper bound on the following probability

$$p_c = \mathbb{P}(I_{1,1} = t_{\text{in}}^1 \wedge I_{1,2} = t_{\text{in}}^2 \wedge I_{2,1} = t_{\text{in}}^3 | O = t_{\text{out}}).$$

From the law of total probability and the Bayes formula, we have

$$\begin{aligned} p_c &= \sum_{i=0}^n \mathbb{P}((I_{1,1} = t_{\text{in}}^1 \wedge I_{1,2} = t_{\text{in}}^2 \wedge I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i) | O = t_{\text{out}}) \\ &= \sum_{i=0}^n \mathbb{P}((I_{1,1} = t_{\text{in}}^1 \wedge I_{1,2} = t_{\text{in}}^2) | (O = t_{\text{out}} \wedge I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i)) \\ &\quad \cdot \mathbb{P}((I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i) | O = t_{\text{out}}) \end{aligned}$$

We now introduce the random variable $\mathcal{Y}$ which represents the subset of output shares from G_1 that are required for the subsequent simulation:

$$\begin{aligned} p_c &= \sum_{i=0}^n \sum_{Y \subseteq [0, n]} \mathbb{P}((I_{1,1} = t_{\text{in}}^1 \wedge I_{1,2} = t_{\text{in}}^2) | (O = t_{\text{out}} \wedge I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i \wedge \mathcal{Y} = Y)) \\ &\quad \cdot \mathbb{P}((\mathcal{Y} = Y) | O = t_{\text{out}} \wedge I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i) \\ &\quad \cdot \mathbb{P}((I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i) | O = t_{\text{out}}) \end{aligned}$$

The events $I_{1,1} = t_{\text{in}}^1$ and $I_{1,2} = t_{\text{in}}^2$ in the first term depend only on the output set $\mathcal{Y}$ (and its cardinality) and not on the subsequent leakage anymore, hence we can simplify the expression:

$$\begin{aligned} p_c &= \sum_{i=0}^n \sum_{\substack{Y \subseteq [0, n] \\ |Y|=i}} \mathbb{P}((I_{1,1} = t_{\text{in}}^1 \wedge I_{1,2} = t_{\text{in}}^2) | (\mathcal{Y} = Y)) \\ &\quad \cdot \mathbb{P}((\mathcal{Y} = Y) | O = t_{\text{out}} \wedge I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i) \\ &\quad \cdot \mathbb{P}((I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i) | O = t_{\text{out}}) \end{aligned}$$

Algorithm 1 RPZeroEnc^γ

Output: $\llbracket z \rrbracket = (z_1, \dots, z_n) \in \mathbb{K}^n$ such that $z_1 + \dots + z_n = 0$

1. $\llbracket z \rrbracket = (0, 0, \dots, 0)$ $\{n \text{ zeros}\}$
2. **for** $i = 1$ to γ **do**
3. Select two distinct indices $i_1, i_2 \in [1, n]$ uniformly at random
4. $r \xleftarrow{\$} \mathbb{K}$
5. $z_{i_1} \leftarrow z_{i_1} + r$
6. $z_{i_2} \leftarrow z_{i_2} - r$
7. **end for**
8. **return** $\llbracket z \rrbracket$

Algorithm 2 $\text{RPRefresh}^\gamma(\llbracket a \rrbracket)$

Input: $\llbracket a \rrbracket = (a_1, \dots, a_n) \in \mathbb{K}^n$

Output: $\llbracket s \rrbracket = (s_1, \dots, s_n) \in \mathbb{K}^n$ such that $s_1 + \dots + s_n = a_1 + \dots + a_n$

1. $\llbracket z \rrbracket \leftarrow \text{RPZeroEnc}^\gamma()$
2. **for** $i = 1$ to n **do**
3. $s_i \leftarrow a_i + z_i$
4. **end for**
5. **return** $\llbracket s \rrbracket$

To obtain an upper bound, we use the worst-case scenario for the first term, which corresponds to the envelope function of G_1 , and we bound the third term using the envelope function of G_2 , denoted by $\mathcal{E}_{t_{\text{out}}}^2(t_{\text{in}}^3, i)$:

$$p_c \leq \sum_{i=0}^n \mathcal{E}_i^1(t_{\text{in}}^1, t_{\text{in}}^2) \cdot \mathcal{E}_{t_{\text{out}}}^2(t_{\text{in}}^3, i) \cdot \sum_{\substack{Y \subseteq [0, n] \\ |Y|=i}} \mathbb{P}((Y = Y) | O = t_{\text{out}} \wedge I_{2,1} = t_{\text{in}}^3 \wedge I_{2,2} = i)$$

The last summation term equals one, which concludes the proof. $\square$

4 New Random Probing Composable Gadgets

Now that we have defined our new security notions, we introduce cardinal-RPC *base* gadgets, which will be used to construct random probing secure circuits. We begin with our new refresh algorithm in Section 4.1, which is based on a novel zero encoding and will serve as the foundation for our elementary linear operations – addition, multiplication by a constant, and copy – detailed in Section 4.2

4.1 A New Versatile Cardinal-RPC Refresh Gadget

Definition 9 (ZeroEncoding). *Let $n \in \mathbb{N}$. A zero-encoding algorithm, denoted ZeroEnc , is a probabilistic algorithm sampling an n -sharing of 0. In other words, $\llbracket z \rrbracket \leftarrow \text{ZeroEnc}()$ such that $\sum z_i = 0$ (with probability 1).*

We here introduce our new refresh gadget, referred to as RPRefresh , for *random-pair refresh*, which is described in Algorithm 2. This gadget is based on the general construction which consists in refreshing the input sharing by the addition of a sharing of 0. The novelty of our refresh gadget lies in the underlying gadget RPZeroEnc to generate a sharing of 0, which is detailed in Algorithm 1. This gadget is parameterized by a number γ of iterations, which influences both its complexity and its security (up to a certain limit).

The process begins with the initialization of a constant all-0 sharing. During each iteration, two share indices are generated uniformly at random⁴ and a fresh random value is respectively added to and subtracted from the corresponding shares. After all iterations have been completed, the sharing of zero produced by RPZeroEnc is added to the input value in RPRefresh , share by share.

While its conceptual idea is simple, this refreshing technique never outputs a perfectly fresh sharing of zero because of the probabilistic nature of the refreshed indexes. In a first step, let us discuss the quality of the output while leaving probing security apart.

⁴ We deviate here from the classical circuit model by introducing the generation of random indices. However, this can be interpreted as sampling a specific circuit (instantiated with our uniform indices) that we fully disclose to the attacker.

Definition 10 (Partition ZeroEncoding). For $P = (n_1, \dots, n_\ell) \in \mathcal{P}(n)$ an integer partition, we define $\text{PartitionZeroEnc}(P)$ as a particular type of distribution producing a sharing $\llbracket z \rrbracket = \llbracket z_1 \rrbracket \parallel \dots \parallel \llbracket z_\ell \rrbracket$ where $\llbracket z_i \rrbracket$ is a fresh n_i -sharing of 0 for every $i \in [1, \ell]$. In other words,

$$\text{PartitionZeroEnc}(n_1, \dots, n_\ell) = \underbrace{z_1, \dots, z_{n_1}}_{\sum z_i=0} \underbrace{z_{n_1+1}, \dots, z_{n_1+n_2}}_{\sum z_i=0} \dots \underbrace{z_{n-n_\ell+1}, \dots, z_n}_{\sum z_i=0}.$$

The two extreme cases in Definition 10 are (worst case) for $P = (1, \dots, 1)$ where $\text{PartitionZeroEnc}(1, \dots, 1) = (0, \dots, 0)$ (the poorest output quality), and (ideal case) for the singleton $P = (n)$ where $\text{PartitionZeroEnc}(n)$ is a uniform sharing of 0 (the best output quality).

Lemma 5. Let $n, \gamma \in \mathbb{N}$. Let ρ be a random tape. There exists a partition $P_{\gamma, \rho} \in \mathcal{P}(n)$, a permutation function $\sigma_{\gamma, \rho} : [1, n] \rightarrow [1, n]$ and another random tape $\rho'_{\gamma, \rho}$ such that

$$\text{RPZeroEnc}^\gamma(\rho) = \sigma_{\gamma, \rho}(\text{PartitionZeroEnc}(P_{\gamma, \rho}, \rho'_{\gamma, \rho}))$$

where RPZeroEnc is depicted in Algorithm 1.

This lemma follows from the definition of RPZeroEnc . Let $\gamma \in \mathbb{N}$ and the random tape ρ be fixed parameters. Here is an intuition of an iterative construction of the partition, denoted P , and the permutation, denoted σ . Before the first iteration, the variable $\llbracket z \rrbracket = (0, \dots, 0)$ is exactly the output of

$$\text{RPZeroEnc}(\underbrace{1, 1, \dots, 1}_{n \text{ times}}).$$

Suppose now that the generated indices in the first iteration are $(i_1, i_2) = (1, 2)$, then we have $\llbracket z \rrbracket = (r_1, -r_1, \dots, 0)$ for $r_1 \leftarrow \$$ which is identically distributed to

$$\text{RPZeroEnc}(2, \underbrace{1, \dots, 1}_{n-2 \text{ times}}).$$

If the generated indices were different from $(1, 2)$, we would have a distribution identical to $\sigma(\text{PartitionZeroEnc}(2, 1, \dots, 1))$ for the same partition but with some coordinate permutation σ . Now for the second iteration, let $r_2 \leftarrow \$$,

$$\begin{aligned} \text{if } (i_1, i_2) &= (1, 2), \llbracket z \rrbracket = (r_1 + r_2, r_1 - r_2, 0 \dots, 0), \\ \text{if } (i_1, i_2) &= (2, 3), \llbracket z \rrbracket = (r_1, -r_1 + r_2, r_2, 0 \dots, 0), \\ \text{if } (i_1, i_2) &= (3, 4), \llbracket z \rrbracket = (r_1, -r_1, r_2, -r_2, 0 \dots, 0). \end{aligned}$$

In these cases, $\llbracket z \rrbracket$ will be statistically indistinguishable to

$$\begin{aligned} &\text{RPZeroEnc}(2, \underbrace{1, \dots, 1}_{n-2 \text{ times}}) \text{ if } (i_1, i_2) = (1, 2), \\ &\text{RPZeroEnc}(3, \underbrace{1, \dots, 1}_{n-3 \text{ times}}) \text{ if } (i_1, i_2) = (2, 3), \\ &\text{RPZeroEnc}(2, 2, \underbrace{1, \dots, 1}_{n-4 \text{ times}}) \text{ if } (i_1, i_2) = (3, 4). \end{aligned}$$

All the cases are captured with a coordinate permutation in a similar way. On and on, one can construct a partition iteratively depending on the randomness of RPZeroEnc . The permutation σ captures the possibility of RPZeroEnc shuffling the order of the refreshed indices. As the number of iterations accumulates, the distribution of $\llbracket z \rrbracket$ becomes increasingly less partitioned, approaching the ideal partition $P = (n)$. This iterative construction is formalized in a more general case (in the presence of leakage) in Lemma 6.

Experiment 4: Random Probing Partition

$\mathcal{W} \leftarrow \text{LeakingWires}(\text{ZeroEnc}, p)$ $\rho \leftarrow \$$ $\llbracket z \rrbracket \leftarrow \text{ZeroEnc}(\rho)$ $\text{leak} \leftarrow \text{AssignWires}(\text{ZeroEnc}, \rho, \mathcal{W}, \emptyset)$ $\llbracket w \rrbracket \leftarrow \text{Sim}(\mathcal{W}, \text{leak})$ $(P, \sigma) := \phi(\mathcal{W})$ $\llbracket u \rrbracket \leftarrow \sigma(\text{PartitionZeroEnc}(P))$ <i>return</i> $(P, \llbracket z \rrbracket, \llbracket w \rrbracket, \llbracket u \rrbracket)$	Drawing the leaking wires. Drawing a random tape. Drawing $\llbracket z \rrbracket$ an encoding of 0. Computing the leaked values. The leakages are given to the simulator. It outputs an encoding of 0 whose randomness is "weakened" due to the leakage. Computing a partition and a permutation corresponding to the leaking wires. Drawing an encoding of 0 corresponding to the "remaining" randomness to compensate the randomness that was given in the leakage.
---	--

Let us now address the random probing security of RPZeroEnc . We will show below in Theorem 1 that RPZeroEnc is cardinal-RPC (we recall that this security notion was presented in Definition 6). To be able to establish the cardinal-RPC property, one needs to take into account the possible leaked iterations in the quality of the refreshing. Naturally, the more leaked randomness, the less qualitative the refreshing is. We introduce the intermediary notion of *random probing partitioned* (RPP) to characterize this quality with respect to the number of iterations and the leakage probability. We later prove the cardinal-RPC security of the refresh (Theorem 1) from the RPP security of the zero-encoding gadget.

Definition 11 (Random-Probing-Partition). Let $n \in \mathbb{N}$ and $\mathcal{P}(n)$ be the set of partitions of n . Let $\mathcal{E}$ be a distribution envelope over $\mathcal{P}(n)$. An n -share zero-encoding gadget $\text{ZeroEnc} : \emptyset \rightarrow \mathbb{K}^n$ is $(p, \mathcal{E})$ -random-probing-partitioned (RPP) for some $p \in [0, 1]$ if there exists a deterministic algorithm ϕ and a PPT simulator Sim such that the outputs of the random experiment 4 satisfy:

1. $P \lesssim \mathcal{E}$, and
2. $\llbracket z \rrbracket$ and $\llbracket w \rrbracket + \llbracket u \rrbracket$ are identically distributed.

Intuition of Definition 11. Note that Definition 11 is different from other security notions introduced in this paper. It does not ensure that the random probing leakage can be simulated. The simulator is instead given the leakage and the leaking wires. The idea of Definition 11 is to show that the output of ZeroEnc is such that

$$\llbracket z \rrbracket \approx \underbrace{\llbracket w \rrbracket}_{\text{leaked 0-encoding}} + \underbrace{\llbracket u \rrbracket}_{\text{fresh partitioned 0-encoding}}.$$

The sharing $\llbracket u \rrbracket$ hence captures the “remaining” randomness of the output sharing $\llbracket z \rrbracket$ given the internal leakage of the gadget. Let us further stress that $\llbracket u \rrbracket$ follows the random distribution $\sigma(\text{PartitionZeroEnc}(P))$ which randomness is highly related to the partition P . The envelope $\mathcal{E}$ upper-bounds the distribution of the partition P defined by $(P, \sigma) = \phi(\text{LeakingWires}(\text{ZeroEnc}, p))$.

The rationale for characterizing RPZeroEnc under the RPP notion is the following. Assume that, after a number of non-leaking iterations, we get a distribution $\llbracket z \rrbracket = \sigma(\text{PartitionZeroEnc}(P))$ for some partition P and coordinate permutation σ . If, during a subsequent iteration, the value of the random r leaks, the output $\llbracket z' \rrbracket$ of this iteration can be expressed as: $\llbracket z' \rrbracket = \llbracket z \rrbracket + \llbracket w \rrbracket$ where $\llbracket w \rrbracket$ is a vector with one coordinate set to r , one to $-r$, and the rest to 0. Given the leakage of r , one can perfectly simulate this $\llbracket w \rrbracket$. This gives the

intuition behind the RPP property of **RPZeroEnc**: non-leaking iterations contribute to reducing the partition structure of $\llbracket u \rrbracket = \sigma(\text{PartitionZeroEnc}(P))$, while leaking iterations result in updates to the $\llbracket w \rrbracket$ component, accumulating the leaked randomness. We note that the leakage can also concern z_{i_1} or z_{i_2} which results in an update of both $\llbracket u \rrbracket$ (or (P, σ)) and $\llbracket w \rrbracket$.

Before formally establishing the RPP security of **RPZeroEnc** in the next lemma, we introduce the concept of *transition set* of a partition. Assume the output of an iteration corresponds to a partition P . The transition set $\mathcal{T}_j(P)$ contains all partitions P' that can be obtained after an iteration with $i_1 = i_2 = j$ while the transition set $\mathcal{T}_{j_1, j_2}(P)$ contains all partitions P' that can be obtained when $i_1 = j_1 \neq i_2 = j_2$. These transition sets are identified in Table 1 and Table 2, considering the different possible scenarios in terms of leakage. In these tables, expressions such as $(z_{i_1} \oplus z_{i_2}) \wedge \bar{r}$ in the leakage column should be interpreted as a scenario where either z_{i_1} or z_{i_2} leaks (but not both) and r does not leak.

Lemma 6 (RPZeroEnc is RPP). *Let n, γ in $\mathbb{N}$ and $p \in [0, 1]$. The n -share zero-encoding gadget RPZeroEnc^γ is $(p, \mathcal{E}^\gamma)$ -RPP for some distribution envelope $\mathcal{E}^\gamma$ defined over $\mathcal{P}(n)$ constructively as follows. For $\gamma = 0$, we simply have $\mathcal{E}^0(\mathbf{1}) = 1$ and $\mathcal{E}^0(P) = 0$ for all $P \in \mathcal{P}(n) \setminus \{\mathbf{1}\}$, where $\mathbf{1} = (1, 1, \dots, 1)$. Then, for all $\gamma > 0$, we define $\mathcal{E}^{\gamma+1}$ from $\mathcal{E}^\gamma$ as follows:*

1. for all $P \in \mathcal{P}(n)$, we initiate $\mathcal{E}^{\gamma+1}(P)$ to 0,
2. for all $P \in \mathcal{P}(n)$, for all $j \in \{1, \dots, |P|\}$, for all $P' \in \mathcal{T}_j(P)$ from Table 1,

$$\mathcal{E}^{\gamma+1}(P') = \mathcal{E}^{\gamma+1}(P') + \mathbb{P}(P \rightarrow P') \cdot \mathcal{E}^\gamma(P),$$

3. for all $P \in \mathcal{P}(n)$, for all $j_1, j_2 \in \{1, \dots, |P|\}$ with $j_1 < j_2$, for all $P' \in \mathcal{T}_{j_1, j_2}(P)$ from Table 2,

$$\mathcal{E}^{\gamma+1}(P') = \mathcal{E}^{\gamma+1}(P') + \mathbb{P}(P \rightarrow P') \cdot \mathcal{E}^\gamma(P).$$

Proof. The **RPZeroEnc** algorithm begins with a shared vector initialized to $(0, \dots, 0)$. Since all shares are constant and known, and based on Definition 11, the only possible partition with strictly positive probability is the vector of ones.

At iteration $\gamma + 1$, two indices i_1 and i_2 are drawn uniformly at random from $\{1, \dots, n\}$. These indices can either belong to the same part n_j of a partition $P = (n_1, n_2, \dots, n_k)$ with probability

$$p_s = \frac{n_j \cdot (n_j - 1)}{n(n - 1)}$$

or to two different parts n_{j_1} and n_{j_2} with probability

$$p_d = 2 \cdot \frac{n_{j_1} \cdot n_{j_2}}{n(n - 1)}.$$

Next, a random value r is drawn uniformly from $\mathbb{K}$ and added to the two elements z_{i_1} and z_{i_2} . If neither of these elements leaks and both indices belong to the same part n_j , then the partition remains unchanged. However, if all these three elements are leaking and both indices belong to the same part n_j , then z_{i_1} and z_{i_2} are excluded from their zero encodings, forming two new independent parts. This scenario corresponds to the first row of Table 1 and occurs with probability

$$p_s \cdot p^2 \cdot (1 - (1 - p)^3),$$

where z_{i_1} and z_{i_2} each leak with probability p , and the value r , being manipulated three times, leaks with probability $1 - (1 - p)^3$.

All possible leakage scenarios, whether the indices are drawn from the same part or from different parts, are detailed in Tables 1 and 2. This demonstrates the construction of our envelope. $\square$

Table 1: Intermediate probabilities for Lemma 6, starting from a partition $P = (n_1, \dots, n_k)$ and when the drawn indices i_1 and i_2 belong to the same subset of cardinal n_j where $p_s := \frac{n_j \cdot (n_j - 1)}{n(n-1)}$.

Leakage	New partition $P' \in \mathcal{T}_j(P)$	Probability $\mathbb{P}(P \rightarrow P')$
$z_{i_1} \wedge z_{i_2} \wedge r$	$(1, 1, n_1, \dots, n_{j-1}, n_j - 2, n_{j+1}, \dots, n_k)$	$p_s \cdot p^2 \cdot (1 - (1-p)^3)$
$z_{i_1} \wedge z_{i_2} \wedge \bar{r}$	$(2, n_1, \dots, n_{j-1}, n_j - 2, n_{j+1}, \dots, n_k)$	$p_s \cdot p^2 \cdot (1-p)^3$
$(z_{i_1} \oplus z_{i_2}) \wedge r$	$(1, n_1, \dots, n_{j-1}, n_j - 1, n_{j+1}, \dots, n_k)$	$2 \cdot p_s \cdot (p - p^2) \cdot (1 - (1-p)^3)$
$((z_{i_1} \oplus z_{i_2}) \wedge \bar{r}) \vee (\bar{z}_{i_1} \wedge \bar{z}_{i_2})$	$(n_1, \dots, n_k)$	$2 \cdot p_s \cdot p \cdot (1-p)^4 + p_s \cdot (1-p)^2$

Table 2: Intermediate probabilities for Lemma 6, starting from a partition $P = (n_1, \dots, n_k)$ and when the drawn indices i_1 and i_2 belong to two blocks of cardinals n_{j_1} and n_{j_2} , where $p_d := 2 \cdot \frac{n_{j_1} \cdot n_{j_2}}{n(n-1)}$.

Leakage	New partition $P' \in \mathcal{T}_{j_1, j_2}(P)$	Probability $\mathbb{P}(P \rightarrow P')$
$z_{i_1} \wedge z_{i_2} \wedge r$	$(1, 1, n_1, \dots, n_{j_1-1}, n_{j_1} - 1, n_{j_1+1}, \dots, n_{j_2-1}, n_{j_2} - 1, n_{j_2+1}, \dots, n_k)$	$p_d \cdot p^2 \cdot (1 - (1-p)^3)$
$z_{i_1} \wedge z_{i_2} \wedge \bar{r}$	$(2, n_1, \dots, n_{j_1-1}, n_{j_1} - 1, n_{j_1+1}, \dots, n_{j_2-1}, n_{j_2} - 1, n_{j_2+1}, \dots, n_k)$	$p_d \cdot p^2 \cdot (1-p)^3$
$z_{i_1} \wedge \bar{z}_{i_2} \wedge r$	$(1, n_1, \dots, n_{j_1-1}, n_{j_1} - 1, n_{j_1+1}, \dots, n_k)$	$p_d \cdot p \cdot (1-p) \cdot (1 - (1-p)^3)$
$\bar{z}_{i_1} \wedge z_{i_2} \wedge r$	$(1, n_1, \dots, n_{j_2-1}, n_{j_2} - 1, n_{j_2+1}, \dots, n_k)$	$p_d \cdot p \cdot (1-p) \cdot (1 - (1-p)^3)$
$z_{i_1} \wedge \bar{z}_{i_2} \wedge \bar{r}$	$(n_1, \dots, n_{j_1-1}, n_{j_1} - 1, n_{j_1+1}, \dots, n_{j_2-1}, n_{j_2} + 1, n_{j_2+1}, \dots, n_k)$	$p_d \cdot p \cdot (1-p)^4$
$\bar{z}_{i_1} \wedge z_{i_2} \wedge \bar{r}$	$(n_1, \dots, n_{j_1-1}, n_{j_1} + 1, n_{j_1+1}, \dots, n_{j_2-1}, n_{j_2} - 1, n_{j_2+1}, \dots, n_k)$	$p_d \cdot p \cdot (1-p)^4$
$\bar{z}_{i_1} \wedge \bar{z}_{i_2} \wedge r$	$(n_1, \dots, n_k)$	$p_d \cdot (1-p)^2 \cdot (1 - (1-p)^3)$
$\bar{z}_{i_1} \wedge \bar{z}_{i_2} \wedge \bar{r}$	$(n_1, \dots, n_{j_1-1}, n_{j_1} + 1, \dots, n_{j_2-1}, n_{j_2} + n_{j_1}, n_{j_2+1}, \dots, n_k)$	$p_d \cdot (1-p)^5$

In order to simplify the security analysis of RPPRefresh, we now introduce an artificial extension of RPZeroEnc, called RPZeroEnc₊, in which all the output shares are individually manipulated after γ iterations. In terms of leakage, this means that all the output shares of RPZeroEnc can now leak with probability p , in addition to the internal leakage already accounted for in Lemma 6.

Lemma 7 (RPZeroEnc₊ is RPP). *Let n, γ in $\mathbb{N}$ and $p \in [0, 1]$. The n -share zero-encoding gadget RPZeroEnc₊ ^{n, γ} is $(p, \mathcal{E}^\gamma)$ -RPP for some distribution envelope $\mathcal{E}^\gamma$ defined over $\mathcal{P}(n)$ constructively as follows. The distribution envelope $\mathcal{E}^\gamma$ is initialized to 0 for all the possible partitions. Then, for all k , for all partition $P = (n_1, \dots, n_k)$ in $\mathcal{P}(n)$, for all $(d_1, \dots, d_k)$ such that $d_1 \leq n_1, \dots, d_k \leq n_k$,*

$$\mathcal{E}^\gamma(P') = \mathcal{E}^\gamma(P) + \mathcal{E}_{\text{RPZeroEnc}}^\gamma(P) \cdot \prod_{i=1}^k \binom{n_i}{d_i} \cdot p^{d_i} \cdot (1-p)^{n_i-d_i}$$

with $\mathcal{E}_{\text{RPZeroEnc}}^\gamma$ the distribution envelope for the RPP property of RPZeroEnc given in Lemma 6 and $P' = (1, \dots, 1, n_1 - d_1, \dots, n_k - d_k)$.

Proof. Let us consider a partition $P = (n_1, \dots, n_k)$ for some $k \in [1, n]$. We recall that the elements n_i for $i \in [1, k]$ represent the cardinals of shares of $\llbracket z \rrbracket$ which form a perfect zero-encoding given the probed wires.

We begin by computing the intermediate probability p_{int} that for each part n_i of P , exactly $d_i (\leq n_i)$ shares of $\llbracket z \rrbracket$ leak. Since each share of $\llbracket z \rrbracket$ leaks independently with probability p , we apply the binomial distribution to obtain:

$$p_{\text{int}} = \prod_{i=1}^k \binom{n_i}{d_i} \cdot p^{d_i} \cdot (1-p)^{n_i-d_i}.$$

As a result, from partition P , we derive a new partition $P' = (1, \dots, 1, n_1 - d_1, \dots, n_k - d_k)$ with probability p_{int} .

Since the probability of obtaining P at the output of RPZeroEnc^γ is independent of p_{int} , the joint probability is simply the product of these two probabilities.

Finally, the probability of obtaining P' at the output of $\text{RPZeroEnc}_+^{n,\gamma}$ is the sum of the probabilities to have a partition P at the output of RPZeroEnc^γ ($\mathcal{E}_{\text{RPZeroEnc}}^\gamma(P)$) and a final leakage that transform P into P' . $\square$

We now introduce the major statement of this section.

Theorem 1 (RPRefresh is Cardinal-RPC). *Let $n, \gamma \in \mathbb{N}$, $p \in [0, 1]$. The n -share gadget RPRefresh^γ is $(p, (\mathcal{E}_t)_{t \in [0, n]})$ -cardinal-RPC with $(\mathcal{E}_t)_{t \in [0, n]}$ defined over $[0, n]$ as follows. For all $t_{\text{in}}, t_{\text{out}} \in [0, n]$,*

$$\mathcal{E}_{t_{\text{out}}}(t_{\text{in}}) = \sum_{i=0}^{t_{\text{in}}} q(i, t_{\text{out}}) \cdot \binom{n-i}{t_{\text{in}}-i} \cdot p^{t_{\text{in}}-i} \cdot (1-p)^{n-t_{\text{in}}}$$

where $q(i, t_{\text{out}})$ is defined constructively as follows:

1. For all $i, t_{\text{out}} \in [0, n]$, initialize $q(i, t_{\text{out}}) \leftarrow 0$.
2. For every $P = (n_1, \dots, n_k) \in \mathcal{P}(n)$, for every $d_1 \in [0, n_1], \dots, d_k \in [0, n_k]$, update

$$q(i, t_{\text{out}}) \leftarrow q(i, t_{\text{out}}) + \mathcal{E}_{\text{RPZeroEnc}_+}^\gamma(P) \cdot \frac{\binom{n_1}{d_1} \cdots \binom{n_k}{d_k}}{\binom{n}{t_{\text{out}}}}$$

for $t_{\text{out}} = d_1 + \dots + d_k$, $i = n_1 \cdot \mathbb{1}_{d_1=n_1} + \dots + n_k \cdot \mathbb{1}_{d_k=n_k}$ and $\mathcal{E}_{\text{RPZeroEnc}_+}^\gamma$ taken from Lemma 7.

Intuitively, $q(i, t_{\text{out}})$ upper bounds the probability that i input shares are required to simulate the leakage on the internal variables of RPRefresh (without considering the input shares) along with the given t_{out} output shares.

Proof. From Lemmas 6 and 7, we have the probabilities associated to each partition of zero encoding cardinals at the output of $\text{RPZeroEnc}_+^\gamma$. In the cardinal-RPC property, the attacker is been given t_{out} output shares. When the indices of some of these output shares recover all the indices of a part of the partition at the output of $\text{RPZeroEnc}_+^\gamma$, then the attacker obtains the corresponding input shares. Conversely, if the indices of the output shares given to the attacker do not entirely recover the indices of a part of the partition at the output of $\text{RPZeroEnc}_+^\gamma$, the corresponding input shares are perfectly masked and hence not required for the simulation. We can compute the related probability with the distribution of a multihypergeometric law. Namely, for a partition $P = (n_1, \dots, n_k)$ for some $k \in [1, n]$, given t_{out} output shares, the probability that i input shares are required to simulate the leakage on the internal variables of RPRefresh (without considering the leakage on the input shares) is given by

$$q(i, t_{\text{out}}) = \sum_{\substack{P=(n_1, \dots, n_k) \\ P \in \mathcal{P}(n)}} \mathcal{E}_{\text{RPZeroEnc}_+}^\gamma(P) \sum_{\substack{(d_1, \dots, d_k) \\ d_1 + \dots + d_k = t_{\text{out}} \\ n_1 \cdot \mathbb{1}_{d_1=n_1} + \dots + n_k \cdot \mathbb{1}_{d_k=n_k} = i}} \frac{\binom{n_1}{d_1} \cdots \binom{n_k}{d_k}}{\binom{n}{t_{\text{out}}}}$$

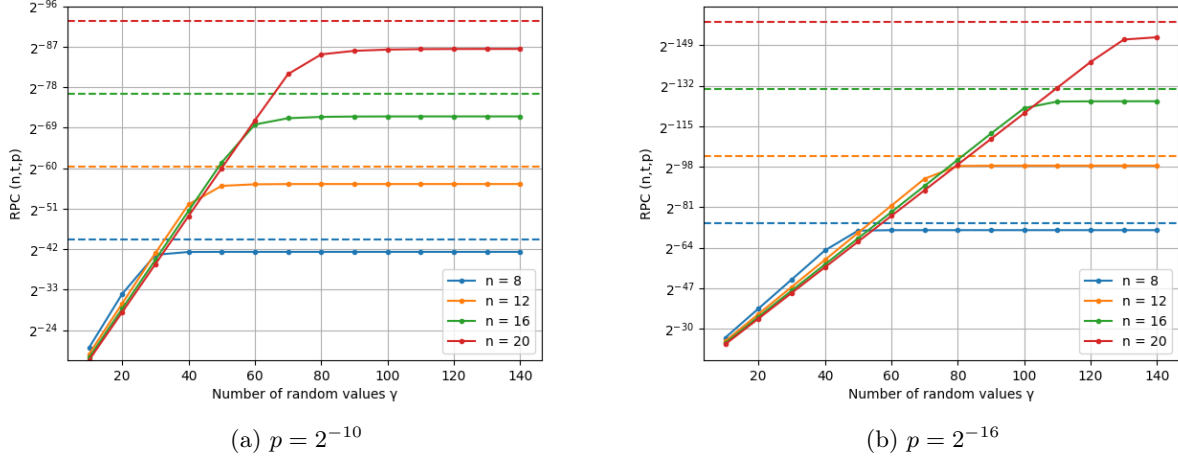


Fig. 3: Computed Threshold-RPC-AI security advantage ε of RPreFresh with fixed $(p, n, \frac{n}{2})$ for $n \in \{8, 12, 16, 20\}$ and $p \in \{p = 2^{-10}, p = 2^{-16}\}$ as a function of the number of iterations γ . The dotted lines corresponds to the maximum achievable security advantage.

in which each d_i represent the number of given output shares whose indices belong to the i^{th} encoding. When $i \leq t_{in}$, this probability is to be multiplied by the probability to recover the remaining input shares according to their leakage. Namely, the probability to recover j input shares among $n - i$ is given by

$$\binom{n-i}{j} \cdot p^j \cdot (1-p)^{n-i-j},$$

which concludes the proof. $\square$

Complexity of the envelope computation. The above results provide explicit formulas to compute the envelopes characterizing the cardinal-RPC security of our refresh. Following Lemma 6, evaluating those formulas has a complexity $\mathcal{O}(\gamma \cdot |\mathcal{P}(n)| \cdot n^2)$ by iterating over the γ iterations of the refresh, the $|\mathcal{P}(n)|$ partitions of n , and the different pairs of elements within a partition (whose number is lower than n^2). According to the Hardy-Ramanujan asymptotic formula, the number of integer partitions of n can be approximated as

$$|\mathcal{P}(n)| \approx \frac{1}{4n\sqrt{3}} \exp\left(\pi\sqrt{\frac{2n}{3}}\right)$$

which is subexponential in n . For instance, $|\mathcal{P}(8)| = 22$, $|\mathcal{P}(16)| = 231$, $|\mathcal{P}(32)| = 8349$, $|\mathcal{P}(64)| < 2^{21}$. In practice, using our (non-optimized) Python implementation of this computation results in, *e.g.*, less than 40 milliseconds for $(n, \gamma) = (8, 50)$, around 9 seconds for $(n, \gamma) = (16, 100)$, around 3 minutes for $(n, \gamma) = (20, 120)$ on a regular laptop.

Figure 3 illustrates the threshold-RPC security advantage of RPreFresh for various numbers of shares n , with $t = \frac{n}{2}$, and for two leakage probabilities, $p = 2^{-10}$ (Figure 3a) and $p = 2^{-16}$ (Figure 3b), as a function of the number of random values γ . Those results have been obtained and validated by two independent implementations (from two different developers) of the envelope evaluation. We observe that, after a sufficient number of random values, the security advantage stabilizes, reaching a steady state. This steady state is logically lower bounded by the probability of observing $t + 1$ input shares from a simple random probing leakage of these shares (i.e., each share leaks with probability p), which is given by $\binom{n}{t+1} \cdot p^{t+1} \cdot (1-p)^{n-t-1}$. In the figure, this probability is represented by dotted lines in colors corresponding to n .

4.2 Simple Sharewise Cardinal-RPC Gadgets

With RPrefresh now defined, we proceed to construct a set of foundational gadgets that will serve as building blocks for many common schemes. The proposed constructions follow a very natural blueprint for defining a sharewise (linear multiplication, addition, or copy) gadget from a refresh gadget, called **Refresh**, such as already proposed in [BRT21, BRTV21]. Our main contribution here is to prove the cardinal-RPC security of these gadgets from that of the underlying refresh.

Linear Multiplication Gadget. The linear multiplication gadget was defined in [BRTV21] with a linear multiplication preceding the refreshing. For consistency with the other simple gadgets, we define it with a refreshing of the shares preceding the linear multiplication with the constant: $G_{\text{cmult}}(\llbracket x \rrbracket, c) = \text{Refresh}(\llbracket x \rrbracket) \cdot c$.

Algorithm 3 $G_{\text{cmult}}(\llbracket x \rrbracket, c)$ adapted from [BRTV21]

Input: $\llbracket x \rrbracket = (x_1, x_2, \dots, x_n)$, constant $c \in \mathbb{K}$
Output: $\llbracket z \rrbracket = (z_1, z_2, \dots, z_n)$ such that $z_1 + z_2 + \dots + z_n = c \cdot (x_1 + x_2 + \dots + x_n)$

1. $\llbracket z \rrbracket \leftarrow \text{Refresh}(\llbracket x \rrbracket)$
2. **for** $i = 1$ to n **do**
3. $z_i \leftarrow z_i \cdot c$
4. **end for**
5. **return** $\llbracket z \rrbracket$

Lemma 8 (G_{cmult} is Cardinal-RPC). *Let $n \in \mathbb{N}$. Let $\text{Refresh}: \mathbb{K}^n \rightarrow \mathbb{K}^n$ be an n -share refresh gadget satisfying $(p, (\mathcal{E}_t^R)_{t \in [0, n]})$ -cardinal-RPC for some collection of envelopes $(\mathcal{E}_t^R)_{t \in [0, n]}$ and some probability $p \in [0, 1]$. Then, G_{cmult} is $(p, (\mathcal{E}_t^{\text{cmult}})_{t \in [0, n]})$ -cardinal-RPC where, for all $t_{\text{out}} \in [0, n]$, $\mathcal{E}_{t_{\text{out}}}^{\text{cmult}}$ is defined, for all $t_{\text{in}} \in [0, n]$, as:*

$$\mathcal{E}_{t_{\text{out}}}^{\text{cmult}}(t_{\text{in}}) = \sum_{i=0}^{n-t_{\text{out}}} \binom{n-t_{\text{out}}}{i} \cdot p^i \cdot (1-p)^{n-t_{\text{out}}-i} \cdot \mathcal{E}_{t_{\text{out}}+i}^R(t_{\text{in}}).$$

Proof. The only additional wires that can potentially leak in G_{cmult} compared to **Refresh** are the output shares of **Refresh**. The probability that t_{in} input shares are required to simulate the leakage and the final output shares is therefore obtained by computing the product of the leakage probability of i such wires with the probability of **Refresh**, considering an output composed of the global output plus these extra leaking wires. The probability that i output shares of **Refresh** leak, out of the $n - t_{\text{out}}^{\text{cmult}}$ output shares that remain unknown to the attacker, is given by

$$\binom{n-t_{\text{out}}}{i} \cdot p^i \cdot (1-p)^{n-t_{\text{out}}-i},$$

which completes the proof. □

Addition Gadget. G_{add} is the simple addition gadget from [BRT21]. It performs $G_{\text{add}}(\llbracket x \rrbracket, \llbracket y \rrbracket) = \text{Refresh}(\llbracket x \rrbracket) + \text{Refresh}(\llbracket y \rrbracket)$.

Algorithm 4 $G_{\text{add}}(\llbracket x \rrbracket, \llbracket y \rrbracket)$ from [BRT21]

Input: $\llbracket x \rrbracket = (x_1, x_2, \dots, x_n)$ and $\llbracket y \rrbracket = (y_1, y_2, \dots, y_n)$
Output: $\llbracket z \rrbracket = (z_1, z_2, \dots, z_n)$ such that $z_1 + z_2 + \dots + z_n = (x_1 + y_1) + (x_2 + y_2) + \dots + (x_n + y_n)$

1. $\llbracket x' \rrbracket \leftarrow \text{Refresh}(\llbracket x \rrbracket)$
2. $\llbracket y' \rrbracket \leftarrow \text{Refresh}(\llbracket y \rrbracket)$
3. **for** $i = 1$ to n **do** $z_i \leftarrow x'_i + y'_i$
4. **return** $\llbracket z \rrbracket$

Lemma 9 (G_{add} is Cardinal-RPC). Let $n \in \mathbb{N}$. Let $\text{Refresh}: \mathbb{K}^n \rightarrow \mathbb{K}^n$ be an n -share refresh gadget satisfying $(p, (\mathcal{E}_t^R)_{t \in [0, n]})$ -cardinal-RPC for some collection envelopes $(\mathcal{E}_t^R)_{t \in [0, n]}$ and some probability $p \in [0, 1]$. Then, G_{add} is $(p, (\mathcal{E}_t^{\text{add}})_{t \in [0, n]})$ -cardinal-RPC where, for all $t_{\text{out}} \in [0, n]$, $\mathcal{E}_{t_{\text{out}}}^{\text{add}}$ is defined, for all $\mathbf{t}_{\text{in}} = (t_{\text{in},1}, t_{\text{in},2}) \in [0, n]^2$, as

$$\begin{aligned} \mathcal{E}_{t_{\text{out}}}^{\text{add}}(\mathbf{t}_{\text{in}}) = & \sum_{i_1=0}^{n-t_{\text{out}}} \binom{n-t_{\text{out}}}{i_1} \cdot p^{i_1} \cdot (1-p)^{n-t_{\text{out}}-i_1} \cdot \mathcal{E}_{t_{\text{out}}+i_1}^R(t_{\text{in},1}) \\ & \cdot \sum_{i_2=0}^{n-t_{\text{out}}} \binom{n-t_{\text{out}}}{i_2} \cdot p^{i_2} \cdot (1-p)^{n-t_{\text{out}}-i_2} \cdot \mathcal{E}_{t_{\text{out}}+i_2}^R(t_{\text{in},2}). \end{aligned}$$

Proof. We assume (this is the worst case) that all the t_{out} output shares given to the attacker require the corresponding $2 \cdot t_{\text{out}}$ output shares of the two instances of Refresh to be simulated. As for G_{cmult} , the output shares to simulate for each instance of Refresh are completed by the possible leakage at the input of the addition. In that case, the probabilities for both instances of Refresh to require $t_{\text{in},\alpha}$ input shares (for $\alpha \in \{1, 2\}$) are independent given the total number of output shares to simulate with the leakage and we obtain the above envelope. $\square$

In general, the addition gadget G_{add} can involve two distinct refresh gadgets, each with a different envelopes collection. Lemma 9 can be easily updated to account for this by getting use of the first envelopes collection in the first sum and the second envelopes collection in the second sum.

Copy Gadget. G_{copy} is the simple copy gadget from [BRT21]. It computes $G_{\text{copy}}(\llbracket x \rrbracket) = (\text{Refresh}(\llbracket x \rrbracket), \text{Refresh}(\llbracket x \rrbracket))$.

Algorithm 5 $G_{\text{copy}}(\llbracket x \rrbracket)$ from [BRT21]

Input: $\llbracket x \rrbracket = (x_1, x_2, \dots, x_n)$

Output: $\llbracket z^1 \rrbracket = (z_1^1, z_2^1, \dots, z_n^1)$ and $\llbracket z^2 \rrbracket = (z_1^2, z_2^2, \dots, z_n^2)$ such that $z_1^1 + z_2^1 + \dots + z_n^1 = z_1^2 + z_2^2 + \dots + z_n^2 = x_1 + x_2 + \dots + x_n$

1. $\llbracket z^1 \rrbracket \leftarrow \text{Refresh}(\llbracket x \rrbracket)$
 2. $\llbracket z^2 \rrbracket \leftarrow \text{Refresh}(\llbracket x \rrbracket)$
 3. **return** $(\llbracket z^1 \rrbracket, \llbracket z^2 \rrbracket)$
-

Lemma 10 (G_{copy} is Cardinal-RPC). Let $n \in \mathbb{N}$. Let $\text{Refresh}: \mathbb{K}^n \rightarrow \mathbb{K}^n$ be an n -share refresh gadget satisfying $(p, (\mathcal{E}_t^R)_{t \in [0, n]})$ -cardinal-RPC for some collection of envelopes $(\mathcal{E}_t^R)_{t \in [0, n]}$ and some probability $p \in [0, 1]$. Then, G_{copy} is $(p, (\mathcal{E}_t^{\text{copy}})_{t \in [0, n]^2})$ -cardinal-RPC where, for all $\mathbf{t}_{\text{out}} \in [0, n]^2$, $\mathcal{E}_{\mathbf{t}_{\text{out}}}^{\text{copy}}$ is defined, for all $t_{\text{in}} \in [0, n]$, as:

$$\mathcal{E}_{\mathbf{t}_{\text{out}}}^{\text{copy}}(t_{\text{in}}) = \sum_{0 \leq i \leq n} \mathcal{E}_{t_{\text{out},1}}^R(i) \cdot \mathcal{E}_{t_{\text{out},2}}^R(t_{\text{in}} - i).$$

Proof. From the property of Refresh , when t_{out}^R output shares are given to the attacker, the probability that t_{in}^R input shares are required for the simulation of both these output shares and the leakage is upper bounded by

$$\mathcal{E}_{t_{\text{out}}}^R(t_{\text{in}}^R).$$

In the context of G_{copy} , we have two instances of Refresh . The probability that t_{in} input shares are required for the simulation of the leakage and the output shares for G_{copy} is thus computed from the probability that one instance of Refresh requires i input shares for its own simulation and the second one $t_{\text{in}} - i$, hence the final result. $\square$

Algorithm 6 $G_{\text{decode}}(\llbracket x \rrbracket)$

Input: $\llbracket x \rrbracket = (x_1, x_2, \dots, x_n)$ **Output:** $x = x_1 + x_2 + \dots + x_n$

1. $\llbracket t \rrbracket \leftarrow \text{Refresh}(\llbracket x \rrbracket)$
 2. $x \leftarrow \text{ShareSum}(\llbracket t \rrbracket)$
 3. **return** x
-

Algorithm 7 $\text{ShareSum}(\llbracket x \rrbracket)$

Input: $\llbracket x \rrbracket = (x_1, x_2, \dots, x_n)$ **Output:** $x = x_1 + x_2 + \dots + x_n$

1. **if** $n = 2$ **then return** $x_1 + x_2$
 2. **if** $n = 3$ **then return** $\text{ShareSum}((x_1, x_2)) + x_3$
 3. $y \leftarrow \text{ShareSum}(x_1, x_2, \dots, x_{\lfloor \frac{n}{2} \rfloor})$
 4. $z \leftarrow \text{ShareSum}(x_{\lfloor \frac{n}{2} \rfloor + 1}, x_{\lfloor \frac{n}{2} \rfloor + 2}, \dots, x_n)$
 5. **return** $x = y + z$
-

The cardinal-RPC advantage of G_{copy} is tighter when the gadget is instantiated with uniformly cardinal-RPC refresh gadgets, as shown in Corollary 1⁵. Similar to the addition gadget, G_{copy} can also be instantiated with two distinct refresh gadgets, and the security results can be easily adapted accordingly.

Corollary 1. *Let $n \in \mathbb{N}$. Let $\text{Refresh}: \mathbb{K}^n \rightarrow \mathbb{K}^n$ be an n -share $(p, (\mathcal{E}_t^R)_{t \in [0, n]})$ -uniformly cardinal-RPC refresh gadget for some collection of envelopes $(\mathcal{E}_t^R)_{t \in [0, n]}$ and some probability $p \in [0, 1]$ (See Definition 16). Then, G_{copy} is $(p, (\mathcal{E}_{\mathbf{t}}^{\text{copy}})_{\mathbf{t} \in [0, n]^2})$ -cardinal-RPC where, for all $\mathbf{t}_{\text{out}} \in [0, n]^2$, $\mathcal{E}_{\mathbf{t}_{\text{out}}}^{\text{copy}}$ is defined as:*

$$\forall t_{\text{in}} \in [0, n], \quad \mathcal{E}_{\mathbf{t}_{\text{out}}}^{\text{copy}}(t_{\text{in}}) = \sum_{\substack{0 \leq i, j \leq n \\ i, j \leq t_{\text{in}} \leq i+j}} \mathcal{E}_{t_{\text{out},1}}^R(i) \cdot \mathcal{E}_{t_{\text{out},2}}^R(j) \cdot \frac{\binom{i}{j-(t_{\text{in}}-i)} \cdot \binom{n-i}{t_{\text{in}}-i}}{\binom{n}{j}}.$$

Proof. We use the property of uniformly cardinal-RPC gadgets for which the sets of input shares required for the simulation are equiprobable for equal cardinals. Specifically, we can compute the probability that two sets of input shares of sizes i and j , required to simulate the leakage of both refresh gadgets and the outputs, represent t_{in} input shares. The number of common elements between both sets follows an hypergeometric law. Namely, the probability to have $i + j - t_{\text{in}}$ common elements is given by

$$\frac{\binom{i}{j-(t_{\text{in}}-i)} \cdot \binom{n-i}{t_{\text{in}}-i}}{\binom{n}{j}},$$

hence the final result. □

4.3 Output Decoding Gadget

An output decoding gadget reconstructs values from their shares and outputs them in plain form. It is involved for variables that should be unmasked before being returned such as the public key in the key generation algorithm or the signature in the signing algorithm. In the probing model, it typically consists of a refresh gadget followed by a straightforward addition of all the shares. To our knowledge, it has not yet been analyzed within the random probing model. In this work, we propose combining a refresh gadget with a recursive addition of the shares (yielding a binary tree of additions), as presented in Algorithm 6.

Since the output of this gadget is intended to be revealed publicly, its security requirements differ from those of the previous gadgets. Specifically, in the context of composition, we are interested in the *cardinal RPC with public output* (Cardinal-RPC-O). This notion is a special case of the Cardinal-RPC-AI-O notion of Definition 7.

Theorem 2 (ShareSum is Cardinal-RPC-O). *Let $n \in \mathbb{N}$. ShareSum from Algorithm 7 is $(p, \mathcal{E})$ -cardinal-RPC-O where $\mathcal{E}$ is defined from the distribution $\mathcal{D}^{(n)}$ over $[0, n]^2$ as:*

⁵ Note that the refresh gadget instantiated in Section 4.1 does not achieve this uniform property. Hence, the tightness of Corollary 1 cannot be obtained in Section 6.

1. Sample $(r, s) \leftarrow \mathcal{D}^{(n)}$
2. Return s

The distribution $\mathcal{D}^{(n)}$ is recursively defined as follows. For every $n > 1$:

1. Sample $(r_1, s_1) \leftarrow \mathcal{D}^{(\lfloor n/2 \rfloor)}$
2. Sample $(r_2, s_2) \leftarrow \mathcal{D}^{(\lceil n/2 \rceil)}$
3. Return

$$(r, s) = \begin{cases} (n, \min(r_1 + r_2, s_1 + \lceil n/2 \rceil, s_2 + \lfloor n/2 \rfloor)) & \text{with proba. } p \\ (r_1 + r_2, \min(r_1 + r_2, s_1 + \lceil n/2 \rceil, s_2 + \lfloor n/2 \rfloor)) & \text{with proba. } 1 - p \end{cases}$$

And for $n = 1$:

$$\begin{cases} \text{Return } (1, 1) & \text{with proba. } p \\ \text{Return } (0, 0) & \text{with proba. } 1 - p \end{cases}$$

Proof. Consider the circuit defined by **ShareSum** which is represented as a binary tree, where the n leaves are the input shares of the circuits and the root the output. Each node leaks with probability p and we want to identify the number of input shares which are necessary for a perfect simulation of the leakage, given that the output x is provided to the simulator. By definition of **ShareSum**, we have the following invariant: an addition node in the tree with n' incoming leaves has two incoming sub-trees, the left one having $\lfloor n'/2 \rfloor$ leaves and the right one having $\lceil n'/2 \rceil$ leaves. The simulation strategy is illustrated on Figure 4.

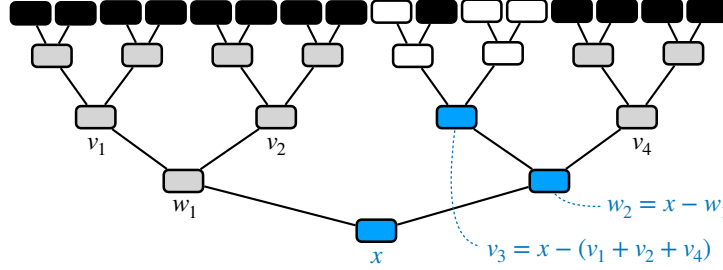


Fig. 4: Illustration of the simulation strategy for the **ShareSum** gadget. White nodes do not leak and hence do not require simulation. Black leaves either leak or are required for the simulation gray nodes. All the gray nodes can be simulated from the leaves (whether they leak or not). All the blue nodes are “simulated from the root”, meaning they are simulated from the output x plus some of the gray nodes. In this example, one needs $n - 3$ input shares for the simulation.

We say that a node can be “simulated from the root” whenever its parent node can be simulated from the root and its sibling node is the root of a sub-tree for which all the leaves are input of the simulation. In the example of Figure 4, w_2 and v_3 satisfy this definition. This makes them simulatable from the root x and some gray nodes which are themselves simulatable from the given leaves. In practice, the pattern represented in Figure 4 can be used for the simulation with any blue path going from the root to one intermediate node (v_3 in the example). The sibling sub-trees of this path (gray nodes in the example) are all fully simulated from their leaves which are required to the simulation. The remaining sub-tree with highest blue node as root is simulated from the required subset of its leaves, depending on the leaking nodes.

To identify the minimal set of input shares required for the simulation one can define a recursive walk from the root to the leaves as follows. Let us denote $T = (R, T_1, T_2)$ any sub-tree with root R , left sub-tree T_1 and right sub-tree T_2 . If $T = (R, \emptyset, \emptyset)$, then R is a leaf. We denote $\text{leak}(R)$ the event that the node R leaks (which occurs with probability p) and $\text{leaves}(T)$ the set of leaves of a sub-tree T . We further denote b ,

a Boolean value indicating whether the root of the current sub-tree T can be simulated from the root (which depends on what happened before in the walk). We obtain the following recursive function:

NeededLeaves(T, b):

1. $(R, T_1, T_2) = T$
2. If $T_1 = T_2 = \emptyset$ (R is a leaf),
 - (a) If $\text{leak}(R)$ (R leaks), return $\{R\}$.
 - (b) If $\neg\text{leak}(R)$ (R does not leak), return $\emptyset$.
3. Let $\mathcal{L}_1 = \text{NeededLeaves}(T_1, 1) \cup \text{leaves}(T_2)$ (make T_2 a gray sub-tree, only possible if $b = 1$).
4. Let $\mathcal{L}_2 = \text{NeededLeaves}(T_2, 1) \cup \text{leaves}(T_1)$ (make T_1 a gray sub-tree, only possible if $b = 1$).
5. Let $\mathcal{L}_3 = \text{NeededLeaves}(T_1, 0) \cup \text{NeededLeaves}(T_2, 0)$ (identify leaves to simulate T_1 and T_2 sub-trees, only possible if R does not leak).
6. If $b = 0$ (R cannot be simulated from the root):
 - (a) If $\text{leak}(R)$, return $\text{leaves}(T)$ (all the leaves are necessary to simulate R).
 - (b) If $\neg\text{leak}(R)$, return $\mathcal{L}_3$ (leaves to simulate T_1 and T_2).
7. If $b = 1$ (R can be simulated from the root): return either $\mathcal{L}_1$, $\mathcal{L}_2$ or $\mathcal{L}_3$, the set with the smallest cardinality.

By calling $\text{NeededLeaves}(T, 1)$ on the full tree T , we obtain the smallest set of leaves that are necessary to simulate all the leaking nodes of the tree according to the simulation strategy exemplified in Figure 4. Consider T as the random variable representing a sub-tree where each node leaks with probability p . Define $\mathcal{R} = \text{NeededLeaves}(T, 0)$ and $\mathcal{S} = \text{NeededLeaves}(T, 1)$. We now argue that the pair $(|\mathcal{R}|, |\mathcal{S}|)$ follows the distribution $\mathcal{D}^{(n_T)}$ defined in Theorem 2, where n_T is the number of leaves of the sub-tree T . The base case is trivial: $(|\mathcal{R}|, |\mathcal{S}|) = (1, 1)$ with probability p (leaf leakage) and $(0, 0)$ otherwise. The general case follows from the description of NeededLeaves , observing that in the definition of $\mathcal{D}^{(n)}$, we have $(r_1, s_1) = (|\mathcal{R}_1|, |\mathcal{S}_1|)$ and $(r_2, s_2) = (|\mathcal{R}_2|, |\mathcal{S}_2|)$ where:

$$(\mathcal{R}_i, \mathcal{S}_i) \sim (\text{NeededLeaves}(T_i, 0), \text{NeededLeaves}(T_i, 1)) \quad \forall i \in \{1, 2\}.$$

□

From Lemma 4, G_{decode} is therefore naturally Cardinal-RPC-O. The distribution $\mathcal{D}^{(n)}$ in the above theorem can be efficiently computed through recursion, providing a precise evaluation of the Cardinal-RPC-O security for our decoding gadget.

5 Random Probing Secure Noise Generation

In this section, we provide a random-probing composable version of the noise generation gadget introduced in Raccoon [dEK⁺23]. In such a context, the base ring of the considered arithmetic circuit is $\mathbb{K} = \mathbb{Z}_q$ for some integer q . The principle of the noise generation gadget is to sample uniform random values over a subset of $\mathbb{Z}_q$ and to add them to the input sharing, thereby introducing the noise required in the underlying Module Learning with Errors (MLWE) instance. In our context, these random values are captured as auxiliary inputs and the noise generation gadget simply aims at summing them to the sharings while minimizing the leakage on these values. We evaluate the security and complexity of two constructions –either in chain or in tree structure– when this sum is instantiated with our new addition gadget introduced in Section 4.2.

5.1 Noise Generation Gadget

Algorithm 8 generically describes the gadget responsible for noise addition. Essentially, it relies on a parsing procedure `DivideAI`, depicted in Algorithm 9, which splits the auxiliary inputs into blocks of n elements from $\mathbb{K}$ and appends zeros to the last block, if necessary. Subsequently, a macro gadget, G_{sum} , is invoked to operate on these blocks of size n . This can be instantiated either with $G_{\text{sum-chain}}$ from Algorithm 10 or with $G_{\text{sum-tree}}$ from Algorithm 11. The input sharing is finally added to the output of G_{sum} to produce the final result.

As stated in Lemma 11, `AddNoiseTo` is immediately cardinal-RPC if both algorithms G_{sum} and G_{add} are cardinal-RPC, as indicated by the composition result in Lemma 4.

Algorithm 8 AddNoiseTo($\llbracket x \rrbracket, \mathbf{a}$)

Input: $\llbracket x \rrbracket = (x_1, x_2, \dots, x_n)$, auxiliary inputs $\mathbf{a} \in \mathbb{K}^\alpha$
Output: $\llbracket y \rrbracket = (y_1, y_2, \dots, y_n)$ such that $y = x + a_1 + a_2 + \dots + a_\alpha$
1. $(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_{\lceil \frac{\alpha}{n} \rceil}) \leftarrow \text{DivideAl}(\mathbf{a}, n)$
2. $\mathbf{z} \leftarrow G_{\text{sum}}(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_{\lceil \frac{\alpha}{n} \rceil})$
3. **return** $G_{\text{add}}(\llbracket x \rrbracket, \mathbf{z})$

Algorithm 9 DivideAl($\mathbf{a}, n$)

Input: Auxiliary inputs $\mathbf{a} \in \mathbb{K}^\alpha, n$
Output: $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_{\lceil \frac{\alpha}{n} \rceil} \in \mathbb{K}^n$
1. **for** $i = 1$ to $\lceil \frac{\alpha}{n} \rceil - 1$ **do**
2. $\mathbf{b}_i \leftarrow (a_{(i-1) \cdot n + 1}, \dots, a_{i \cdot n})$
3. **end for**
4. $\mathbf{b}_{\lceil \frac{\alpha}{n} \rceil} \leftarrow (a_{(\lceil \frac{\alpha}{n} \rceil - 1) \cdot n + 1}, \dots, a_\alpha, 0, \dots, 0)$
5. **return** $(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_{\lceil \frac{\alpha}{n} \rceil})$

Algorithm 10 $G_{\text{sum-chain}}(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_\ell)$

Input: $(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_\ell) \in (\mathbb{K}^n)^\ell$
Output: $\mathbf{z} \in \mathbb{K}^n$
1. $\mathbf{t} \leftarrow \mathbf{b}_1$
2. **for** $i = 2$ to ℓ **do**
3. $\mathbf{t} \leftarrow G_{\text{add}}(\mathbf{t}, \mathbf{b}_i)$
4. **end for**
5. **return** $\mathbf{t}$

Algorithm 11 $G_{\text{sum-tree}}(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_\ell)$

Input: $(\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_\ell) \in (\mathbb{K}^n)^\ell$
Output: $\mathbf{z} \in \mathbb{K}^n$
1. **if** $\ell = 1$ **return** $\mathbf{b}_1$
2. **return** $G_{\text{add}}(G_{\text{sum-tree}}(\mathbf{b}_1, \dots, \mathbf{b}_{\lfloor \ell/2 \rfloor}), G_{\text{sum-tree}}(\mathbf{b}_{\lfloor \ell/2 \rfloor + 1}, \mathbf{b}_2, \dots, \mathbf{b}_\ell))$

Lemma 11 (AddNoiseTo is Cardinal-RPC). Let $n, \alpha \in \mathbb{N}$ and $p \in [0, 1]$. Let $G_{\text{sum}}: (\mathbb{K}^n)^{\lceil \frac{\alpha}{n} \rceil - 1} \rightarrow \mathbb{K}^n$ be $(p, (\mathcal{E}_t^{G_{\text{sum}}})_{t \in [0, n]})$ -cardinal-RPC for some envelopes collection $(\mathcal{E}_t^{G_{\text{sum}}})_{t \in [0, n]}$ defined over $[0, n]^{\lceil \frac{\alpha}{n} \rceil - 1}$ and $G_{\text{add}}: (\mathbb{K}^n)^2 \rightarrow \mathbb{K}^n$ be $(p, (\mathcal{E}_t^{G_{\text{add}}})_{t \in [0, n]})$ -cardinal-RPC for some envelopes collection $(\mathcal{E}_t^{G_{\text{add}}})_{t \in [0, n]}$ defined over $[0, n]^2$. Then, AddNoiseTo, defined in Algorithm 8, is $(p, (\mathcal{E}_t^{ADN})_{t \in [0, n]})$ -cardinal-RPC for some envelopes collection $(\mathcal{E}_t^{ADN})_{t \in [0, n]}$ defined over $[0, n] \times [0, \alpha]$ such that for all $t \in [0, n]$, for all $\mathbf{t}_{in} = (t_{in}^x, t_{in,1}^a, \dots, t_{in, \lceil \frac{\alpha}{n} \rceil}^a) \in [0, n]^{\lceil \frac{\alpha}{n} \rceil + 1}$,

$$\mathcal{E}_t^{ADN}(t_{in}^x, t_{in,1}^a + \dots + t_{in, \lceil \frac{\alpha}{n} \rceil}^a) = \sum_{i=0}^n \mathcal{E}_i^{G_{\text{sum}}}(t_{in,1}^a, \dots, t_{in, \alpha}^a) \cdot \mathcal{E}_t^{G_{\text{add}}}(t_{in}^x, i).$$

In the following, we explore two structures for generating noise: a chain structure, illustrated in Figure 5a, and a binary tree structure, shown in Figure 5b. The respective algorithms for these structures are provided in Algorithms 10 and 11. Lemmas 12 and 13 establish their cardinal-RPC properties.

Lemma 12 is proven by recursively applying the composition result from Lemma 4. Similarly, Lemma 13 is established by applying the same composition result recursively on each branch of the tree, starting from the output nodes.

Lemma 12 ($G_{\text{sum-chain}}$ is Cardinal-RPC). Let $n, \ell \in \mathbb{N}$ and $p \in [0, 1]$. Let $G_{\text{add}}: (\mathbb{K}^n)^2 \rightarrow \mathbb{K}^n$ be an n -share $(p, (\mathcal{E}_t^{G_{\text{add}}})_{t \in [0, n]})$ -cardinal-RPC for some envelopes collection $(\mathcal{E}_t^{G_{\text{add}}})_{t \in [0, n]}$ defined over $[0, n]^2$. Gadget $G_{\text{sum-chain}}: (\mathbb{K}^n)^\ell \rightarrow \mathbb{K}^n$, instantiated with G_{add} , is $(p, (\mathcal{E}_t^{SNC})_{t \in [0, n]})$ -cardinal-RPC for some envelopes collection $(\mathcal{E}_t^{SNC})_{t \in [0, n]}$ defined over $[0, n]^\ell$ as follows:

$$\forall \mathbf{t}_{in} = (t_{in}^1, t_{in}^2, \dots, t_{in}^\ell) \in [0, n]^\ell, \\ \mathcal{E}_t^{SNC}(\mathbf{t}_{in}) = \sum_{i_3=0}^n \mathcal{E}_{i_3}^{G_{\text{add}}}(t_{in}^1, t_{in}^2) \cdot \dots \cdot \sum_{i_\ell=0}^n \mathcal{E}_{i_\ell}^{G_{\text{add}}}(i_{\ell-1}, t_{in}^{\ell-1}) \cdot \mathcal{E}_t^{G_{\text{add}}}(i_\ell, t_{in}^\ell).$$

Lemma 13 ($G_{\text{sum-tree}}$ is Cardinal-RPC). Let $n, \ell \in \mathbb{N}$ and $p \in [0, 1]$. Let $G_{\text{add}}: (\mathbb{K}^n)^2 \rightarrow \mathbb{K}^n$ be an n -share $(p, (\mathcal{E}_t^{G_{\text{add}}})_{t \in [0, n]})$ -cardinal-RPC for some envelopes collection $(\mathcal{E}_t^{G_{\text{add}}})_{t \in [0, n]}$ defined over $[0, n]^2$. Gadget

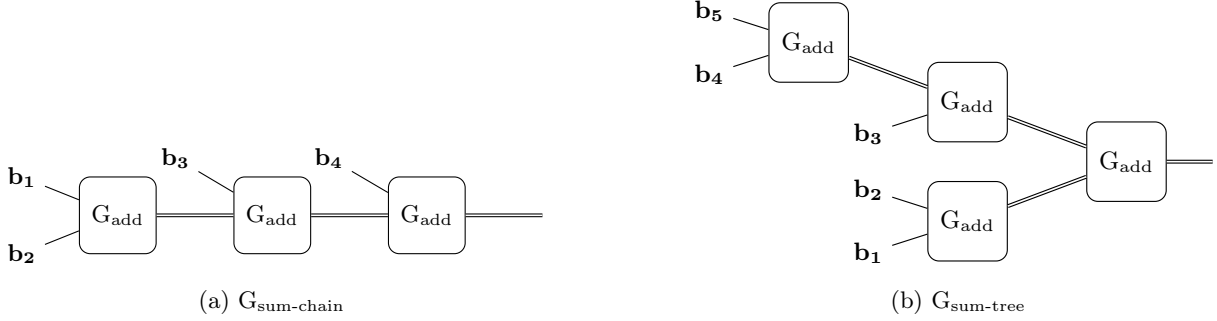


Fig. 5: Noise generation gadget.

$G_{\text{sum-tree}}: (\mathbb{K}^n)^\ell \rightarrow \mathbb{K}^n$, instantiated with G_{add} , is $(p, (\mathcal{E}_t^{SPT(\ell)})_{t \in [0, n]})$ -cardinal-RPC for some envelopes collection $(\mathcal{E}_t^{SPT(\ell)})_{t \in [0, n]}$ recursively defined over $[0, n]^\ell$ as follows. For all $\mathbf{t}_{in} = (t_{in}^1, t_{in}^2, \dots, t_{in}^\ell) \in [0, n]^\ell$,

$$\begin{aligned} \text{if } \ell = 3, \quad \mathcal{E}_t^{SPT(3)}(t_{in}^1, t_{in}^2, t_{in}^3) &= \sum_{i=0}^n \mathcal{E}_i^{G_{\text{add}}}(t_{in,1}, t_{in,2}) \cdot \mathcal{E}_t^{G_{\text{add}}}(i, t_{in,3}) \\ \text{otherwise, } \mathcal{E}_t^{SPT(\ell)}(\mathbf{t}_{in}) &= \sum_{i_1=0}^n \sum_{i_2=0}^n \mathcal{E}_{i_1}^{SPT(\lfloor \ell/2 \rfloor)}(t_{in}^1, \dots, t_{in}^{\lfloor \ell/2 \rfloor}) \cdot \mathcal{E}_{i_2}^{SPT(\lceil \ell/2 \rceil)}(t_{in}^{\lfloor \ell/2 \rfloor + 1}, \dots, t_{in}^\ell) \cdot \mathcal{E}_t^{G_{\text{add}}}(i_1, i_2) \end{aligned}$$

with the sub-tree $SPT(\alpha)$ being $(p, (\mathcal{E}_t^{SPT(\alpha)})_{t \in [0, n]})$ -cardinal-RPC for $3 \leq \alpha \leq n$ for some envelope $(\mathcal{E}_t^{SPT(\alpha)})_{t \in [0, n]}$ and with $SPT(2) = G_{\text{add}}$.

5.2 Instantiation from New Gadgets

In a first attempt, we can use the expansion strategy from [BCP⁺20] to construct a noise generation gadget based on a small addition gadget, as detailed in Appendix B.2. In this section, we illustrate how to leverage our new addition gadget G_{add} to achieve the same objective. Table 3 displays the threshold-RPC advantage of G_{add} from its cardinal-RPC envelopes (see Lemma 9) while fixing the leakage probability, the number of shares, and the value of t . The complexity is assessed in terms of the number of additions and random values required⁶. As anticipated, the complexities for the same leakage probability and security threshold are more favorable than those obtained through the expansion strategy.

Building on these findings, we can evaluate the RPC security of AddRepNoise for specific probabilities and sizes α of the auxiliary input. With cardinal-RPC gadgets, the composition is tighter, and the requirement for auxiliary inputs is considered globally. For example, with $\alpha = 75$ auxiliary inputs and a leakage probability $p = 2^{-24}$, one option is to combine three G_{add} gadgets, each with 25 shares, to set up AddNoiseTo. If we fix the number of iterations γ to 25 for each refresh, the resulting RPC advantage is upper bounded by 2^{-182} both for the chain and the tree structure, when 10 shares from the main input and 30 auxiliary inputs are required for the simulation. As for the complexity, the noise gadget requires 150 random values and implements 525 additions. This is to be compared with the 1200 randoms, 2475 additions and 1200 copies required by the AddNoiseTo gadget obtained from the expansion strategy (and which achieves an RPC advantage of 2^{-169}).

⁶ We omit the count of random values used for selecting indices in the refresh gadget, as they differ in nature from the random produced by the random gates of the circuit. Indeed, these random indices are from $[1, n]$ and not from $\mathbb{K}$. For the considered application (Raccoon), or other lattice-based schemes, elements from $[1, n]$ are expected to be (significantly) smaller than elements from $\mathbb{K}$. Moreover, the random index values in our refresh can be fully revealed to the adversary without compromising the security (which relies on their uniformity but not on their secrecy) unlike the random values of $\mathbb{K}$ produced by random gates which leak with probability p .

Table 3: RPC security advantage and complexity of G_{add} .

Leakage probability p	2^{-12}	2^{-16}	2^{-20}	2^{-24}
# Shares n and # leaking shares t	(25,13)	(20,10)	(16,8)	(12,6)
# Randoms	240	220	220	200
# Additions	555	500	488	436
RPC advantage	2^{-128}	2^{-128}	2^{-134}	2^{-129}

Remark 1. Our experiments show that the tree structure generally provides a slightly better security advantage compared to the chain structure, while maintaining the same single-thread complexity. We believe this is due to the higher average number of dependencies per input in the tree structure, and their balance. Furthermore, both structures offer the potential to explore different instantiations for the refresh gadgets – *i.e.* varying the number of iterations – based on their position, a direction we leave for future research.

6 Random Probing Secure Implementation of Raccoon

We recall the key generation and signature algorithms of Raccoon in Algorithms 12 and 14 from [dEK⁺23]. We refer to [dEK⁺23] for an explicit definition of the subroutines and their rationale. The parameters relevant to our work are presented in Table 4. Specifically, parameters n_R (denoted n in [dEK⁺23]) and q define the quotient ring $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^{n_R} + 1)$, while k and ℓ define the MLWE matrix dimensions. Like in the proof of Raccoon in the d -probing model [dKPR24], we do not consider that the noise samples (r_i) and (r'_i) are generated within AddRepNoise algorithm; instead, they are provided as auxiliary inputs. We also artificially de-correlate the number of shares, here denoted n , from the parameter d , fixed in the specifications.

6.1 Random Probing Secure Construction

We denote by G^{vec} the natural extension of a gadget G , originally operating on elements of $\mathbb{Z}_q$, to a gadget operating on elements of $\mathcal{R}_q$ and which applies the same operation component-wise, repeating it n_R times. We also introduce the following core gadgets:

- G_{cpmult} : This gadget multiplies a uniform public polynomial c_{poly}^i in $\mathcal{R}_q$ with a shared secret vector $\llbracket s^i \rrbracket$ in $(\mathcal{R}_q^n)$, for $i \in [1, \ell]$. It requires n_R^2 n -share linear multiplications on elements of $\mathbb{Z}_q$, $n_R(n_R - 1)$ n -share additions on elements of $\mathbb{Z}_q$ and $n_R(n_R - 1)$ n -share copies on elements of $\mathbb{Z}_q$.
- G_{cmult} : This gadget performs a multiplication between a uniform public matrix A in $\mathcal{R}_q^{k \times \ell}$ with a shared secret vector $\llbracket s \rrbracket$ in $(\mathcal{R}_q^\ell)^n$. It requires $\ell k n_R^2$ n -share linear multiplications on elements of $\mathbb{Z}_q$, $n_R(n_R - 1)\ell k + n_R(\ell - 1)k$ n -share additions on elements of $\mathbb{Z}_q$ and $n_R(n_R - 1)\ell k + n_R(k - 1)\ell$ n -share copies on elements of $\mathbb{Z}_q$.

To obtain a random-probing secure construction, masked gadgets secure in the probing model can be seamlessly replaced by random-probing composable gadgets. The random-probing Raccoon equivalent is illustrated in Algorithms 13 and 15. Let us provide an intuition for the random probing version of the key generation. The original algorithm (Alg. 12) starts with a fresh zero-encoding (line 3) and adds the auxiliary inputs $(r_i)_{1 \leq i \leq \ell}$ with intermediate refreshes (inside AddRepNoise in line 4). In our design, the refresh gadgets are already included in the elementary gadgets, so we simply sum the auxiliary inputs $(r_i)_{1 \leq i \leq \ell}$ split in convenient packs using DivideAI and the G_{sum} gadget (see Section 5). Next, the resulting sharing $\llbracket s \rrbracket$ is copied: one copy is used for the public key and the other one for the secret key. After the multiplication step, we use AddNoiseTo (presented in Alg. 8) to incorporate the auxiliary inputs $(r'_i)_{1 \leq i \leq k}$ and finally decode and output the result. The same reasoning can be similarly applied to the signature algorithm. In a nutshell, compared to the original d -probing secure algorithms, most intermediate refresh gadgets can be removed and copy gadgets are added whenever a variable needs to be reused. We stress that *our random-probing Raccoon is functionally equivalent to the original d -probing version.*

Algorithm 12 d -Probing-KeyGen

Auxiliary Inputs: $(r_i)_{1 \leq i \leq \ell} \in (\{0, 1\}^{u_t})^{d \cdot \text{rep} \cdot \ell \cdot n_R}$
 $(r'_i)_{1 \leq i \leq k} \in (\{0, 1\}^{u_t})^{d \cdot \text{rep} \cdot k \cdot n_R}$ **Output:** Key pair (vk, sk)

1. $\text{seed} \leftarrow \{0, 1\}^\kappa$
 2. $A \leftarrow \text{ExpandA}(\text{seed})$
 3. $\llbracket s \rrbracket \leftarrow \ell \times \text{ZeroEncoding}(d)$
 4. $\llbracket s \rrbracket \leftarrow \text{AddRepNoise}(\llbracket s \rrbracket, (r_i)_{1 \leq i \leq \ell})$
 5. $\llbracket t \rrbracket \leftarrow A \cdot \llbracket s \rrbracket$
 6. $\llbracket t \rrbracket \leftarrow \text{AddRepNoise}(\llbracket t \rrbracket, (r'_i)_{1 \leq i \leq k})$
 7. $t \leftarrow \text{Decode}(\llbracket t \rrbracket)$
 8. **return** $(\text{vk} \leftarrow (\text{seed}, t), \text{sk} \leftarrow (t, \llbracket s \rrbracket))$
-

Algorithm 13 RP-KeyGen

Auxiliary Inputs: $(r_i)_{1 \leq i \leq \ell} \in (\{0, 1\}^{u_t})^{d \cdot \text{rep} \cdot \ell \cdot n_R}$
 $(r'_i)_{1 \leq i \leq k} \in (\{0, 1\}^{u_t})^{d \cdot \text{rep} \cdot k \cdot n_R}$ **Output:** Key pair (vk, sk)

1. $\text{seed} \leftarrow \{0, 1\}^\kappa$
 2. $A \leftarrow \text{ExpandA}(\text{seed})$
 3. $\mathbf{r}_b \leftarrow \text{DivideAI}^{\text{vec}}((r_i)_{1 \leq i \leq \ell})$
 4. $\llbracket s \rrbracket \leftarrow G_{\text{sum}}^{\text{vec}}(\mathbf{r}_b)$
 5. $(\llbracket s \rrbracket, \llbracket s_t \rrbracket) \leftarrow G_{\text{copy}}^{\text{vec}}(\llbracket s \rrbracket)$
 6. $\llbracket t \rrbracket \leftarrow G_{\text{cmmult}}(A, \llbracket s_t \rrbracket)$
 7. $\llbracket t \rrbracket \leftarrow \text{AddNoiseTo}^{\text{vec}}(\llbracket t \rrbracket, (r'_i)_{1 \leq i \leq k})$
 8. $t \leftarrow G_{\text{decode}}^{\text{vec}}(\llbracket t \rrbracket)$
 9. **return** $(\text{vk} \leftarrow (\text{seed}, t), \text{sk} \leftarrow (t, \llbracket s \rrbracket))$
-

Algorithm 14 d -Probing-Sign

Auxiliary Inputs: $(r_i)_{1 \leq i \leq \ell} \in (\{0, 1\}^{u_t})^{d \cdot \text{rep} \cdot \ell \cdot n_R}$
 $(r'_i)_{1 \leq i \leq k} \in (\{0, 1\}^{u_t})^{d \cdot \text{rep} \cdot k \cdot n_R}$ **Input:** $\text{sk} = (\text{vk}, \llbracket s \rrbracket)$, $\text{msg} \in \{0, 1\}^*$ **Output:** $\text{sig} = (c_{\text{hash}}, h, z)$

1. $(\text{vk}, \llbracket s \rrbracket) \leftarrow \text{sk}, (\text{seed}, t) \leftarrow \text{vk}$
 2. $\mu \leftarrow H(H(\text{vk}) \parallel \text{msg})$
 3. $A \leftarrow \text{ExpandA}(\text{seed})$
 4. $\llbracket r \rrbracket \leftarrow \ell \times \text{ZeroEncoding}(d)$
 5. $\llbracket r \rrbracket \leftarrow \text{AddRepNoise}(\llbracket r \rrbracket, (r_i)_{1 \leq i \leq \ell})$
 6. $\llbracket w \rrbracket \leftarrow A \cdot \llbracket r \rrbracket$
 7. $\llbracket w \rrbracket \leftarrow \text{AddRepNoise}(\llbracket w \rrbracket, (r'_i)_{1 \leq i \leq k})$
 8. $w \leftarrow \text{Decode}(\llbracket w \rrbracket)$
 9. $c_{\text{hash}} \leftarrow \text{ChalHash}(w, \mu)$
 10. $c_{\text{poly}} \leftarrow \text{ChalPoly}(c_{\text{hash}})$
 11. $\llbracket s \rrbracket \leftarrow \text{Refresh}(\llbracket s \rrbracket)$
 12. $\llbracket r \rrbracket \leftarrow \text{Refresh}(\llbracket r \rrbracket)$
 13. $\llbracket z \rrbracket \leftarrow c_{\text{poly}} \cdot \llbracket s \rrbracket + \llbracket r \rrbracket$
 14. $\llbracket z \rrbracket \leftarrow \text{Refresh}(\llbracket z \rrbracket)$
 15. $z \leftarrow \text{Decode}(\llbracket z \rrbracket)$
 16. $h \leftarrow w - A \cdot z - c_{\text{hash}} \cdot t$
 17. $\text{sig} \leftarrow (c_{\text{hash}}, h, z)$
 18. **if** $\text{CheckBounds}(\text{sig}) = \text{FAIL}$ **then**
 19. **goto** Line 4
 20. **end if**
 21. **return** sig
-

Algorithm 15 RP-Sign

Auxiliary Inputs: $(r_i)_{1 \leq i \leq \ell} \in (\{0, 1\}^{u_t})^{d \cdot \text{rep} \cdot \ell \cdot n_R}$
 $(r'_i)_{1 \leq i \leq k} \in (\{0, 1\}^{u_t})^{d \cdot \text{rep} \cdot k \cdot n_R}$ **Input:** $\text{sk} = (\text{vk}, \llbracket s \rrbracket)$, $\text{msg} \in \{0, 1\}^*$ **Output:** $\text{sig} = (c_{\text{hash}}, h, z)$

1. $(\text{vk}, \llbracket s \rrbracket) \leftarrow \text{sk}, (\text{seed}, t) \leftarrow \text{vk}$
 2. $\mu \leftarrow H(H(\text{vk}) \parallel \text{msg})$
 3. $A \leftarrow \text{ExpandA}(\text{seed})$
 4. $\mathbf{r}_b \leftarrow \text{DivideAI}^{\text{vec}}((r_i)_{1 \leq i \leq \ell})$
 5. $\llbracket r \rrbracket \leftarrow G_{\text{sum}}^{\text{vec}}(\mathbf{r}_b)$
 6. $(\llbracket r \rrbracket, \llbracket r_c \rrbracket) \leftarrow G_{\text{copy}}^{\text{vec}}(\llbracket r \rrbracket)$
 7. $\llbracket w \rrbracket \leftarrow G_{\text{cmmult}}(A, \llbracket r \rrbracket)$
 8. $\llbracket w \rrbracket \leftarrow \text{AddNoiseTo}^{\text{vec}}(\llbracket w \rrbracket, (r'_i)_{1 \leq i \leq k})$
 9. $w \leftarrow G_{\text{decode}}^{\text{vec}}(\llbracket w \rrbracket)$
 10. $c_{\text{hash}} \leftarrow \text{ChalHash}(w, \mu)$
 11. $c_{\text{poly}} \leftarrow \text{ChalPoly}(c_{\text{hash}})$
 12. $\llbracket z \rrbracket \leftarrow G_{\text{add}}^{\text{vec}}(G_{\text{cpmult}}(\llbracket s \rrbracket, c_{\text{poly}}), \llbracket r_c \rrbracket)$
 13. $z \leftarrow G_{\text{decode}}^{\text{vec}}(\llbracket z \rrbracket)$
 14. $h \leftarrow w - A \cdot z - c_{\text{hash}} \cdot t$
 15. $\text{sig} \leftarrow (c_{\text{hash}}, h, z)$
 16. **if** $\text{CheckBounds}(\text{sig}) = \text{FAIL}$ **then**
 17. **goto** Line 4
 18. **end if**
 19. **return** sig
-

Table 4: Some of Raccoon’s parameters (see [dEK⁺23] for the full description).

Param.	Raccoon-128-16
q	549824583172097
n_R	512
k	5
ℓ	4
d	16
rep	2

Table 5: Parameters introduced for the random probing security.

Param.	Description
p	Probability of leaking a wire (resulting from the noise level of the device).
ϵ	Threshold-RPC-AI-O advantage, indicates the security level (high for weak security, low for strong security).
n	Number of shares ($= d$ or $\geq d$ if expansion, often higher to reach better security levels).
t	Threshold RPC parameter (set to $n/2$).
γ	Number of iterations of the refresh algorithm, allowing optimizations of the security/performance trade-off.

Instantiation with our new gadgets. We now instantiate Algorithms 13 and 15 with our new cardinal-RPC(-AI-O) secure gadgets defined in Section 4.2. Figures 6 and 7 (in Appendix) visually illustrate the processes for Raccoon-128. We denote by G_{copies}^k the combination of copy gadgets arranged in a binary tree structure to produce k copies of the same sharing, and by G_{cpmult}^k the application of the G_{cpmult} gadget to perform a polynomial multiplication on $\mathcal{R}_q$ for k separate inputs.

With these new gadgets, our schemes can be instantiated in various ways, with multiple parameters subject to variation – namely, the probability p , the RPC threshold t , and the number of iterations γ for each refresh instance. All these new parameters are summarized in Table 5. We opt for aligning to the Raccoon-128-16 configuration, which operates with 16-sharings. The corresponding instantiation is detailed in Table 4, while the full set of parameters can be found in [dEK⁺23, Table 2]. The scheme is proven EUF-CMA secure even when up to $d - 1 = 15$ values of r_i and r'_i per polynomial coefficient are exposed to the attacker. In other words, the existing security model assumes that at most 15 out of 32 values of r_i and r'_i can be given to the attacker. Hence, at least 17 small uniforms per polynomial coefficient must remain secret. Note that this applies to each of the $n_R \times \ell$ (or $n_R \times k$ for r'_i) polynomial coefficients. Intuitively, the security then corresponds to the security of Raccoon with $\text{rep} = 1$ instead of 2, denoted “Small Raccoon” in [dKPR24]. We refer to [dKPR24, Security of Small Raccoon in Sec. 7] for more information.

The Raccoon-128-16 configuration is particularly well-suited because using 16-sharings requires two inputs per coefficient to incorporate the small uniforms. This results in a single addition gadget per coefficient in the first G_{sum} and two addition gadgets per coefficient in the AddNoiseTo gadget.

Table 6: Threshold-RPC-AI-O advantages and complexities of Raccoon’s key generation and signature when instantiated with our new gadgets. The values for γ (chosen to be different for block 1, block 2 and G_{cmult}) are fixed to minimize ϵ (which impacts the overall randomness consumption).

Key Generation					Signature				
p		2^{-24}	2^{-20}	2^{-16}	2^{-12}	2^{-24}	2^{-20}	2^{-16}	2^{-12}
		$\epsilon \leq 2^{-128}$				$\epsilon \leq 2^{-128}$			
# additions		$2^{31.5}$	$2^{31.7}$	-	-	$2^{32.1}$	$2^{32.3}$	-	-
# linear mult.		$2^{26.3}$	$2^{26.3}$	-	-	$2^{26.6}$	$2^{26.6}$	-	-
# randoms		$2^{30.3}$	$2^{30.5}$	-	-	$2^{30.8}$	$2^{31.1}$	-	-
		$\epsilon \leq 2^{-80}$				$\epsilon \leq 2^{-80}$			
# additions		$2^{30.7}$	$2^{30.9}$	$2^{31.1}$	-	$2^{31.3}$	$2^{31.4}$	$2^{31.6}$	-
# linear mult.		$2^{26.3}$	$2^{26.3}$	$2^{26.3}$	-	$2^{26.6}$	$2^{26.6}$	$2^{26.6}$	-
# randoms		$2^{29.2}$	$2^{29.5}$	$2^{29.8}$	-	$2^{30.3}$	$2^{30.5}$	$2^{30.7}$	-
		$\epsilon \leq 2^{-64}$				$\epsilon \leq 2^{-64}$			
# additions		$2^{30.4}$	$2^{30.5}$	$2^{30.8}$	$2^{31.0}$	$2^{31.0}$	$2^{31.1}$	$2^{31.3}$	$2^{31.5}$
# linear mult.		$2^{26.3}$	$2^{26.3}$	$2^{26.3}$	$2^{26.3}$	$2^{26.6}$	$2^{26.6}$	$2^{26.6}$	$2^{26.6}$
# randoms		$2^{28.8}$	$2^{29.0}$	$2^{29.3}$	$2^{29.6}$	$2^{29.5}$	$2^{29.7}$	$2^{30.0}$	$2^{30.2}$

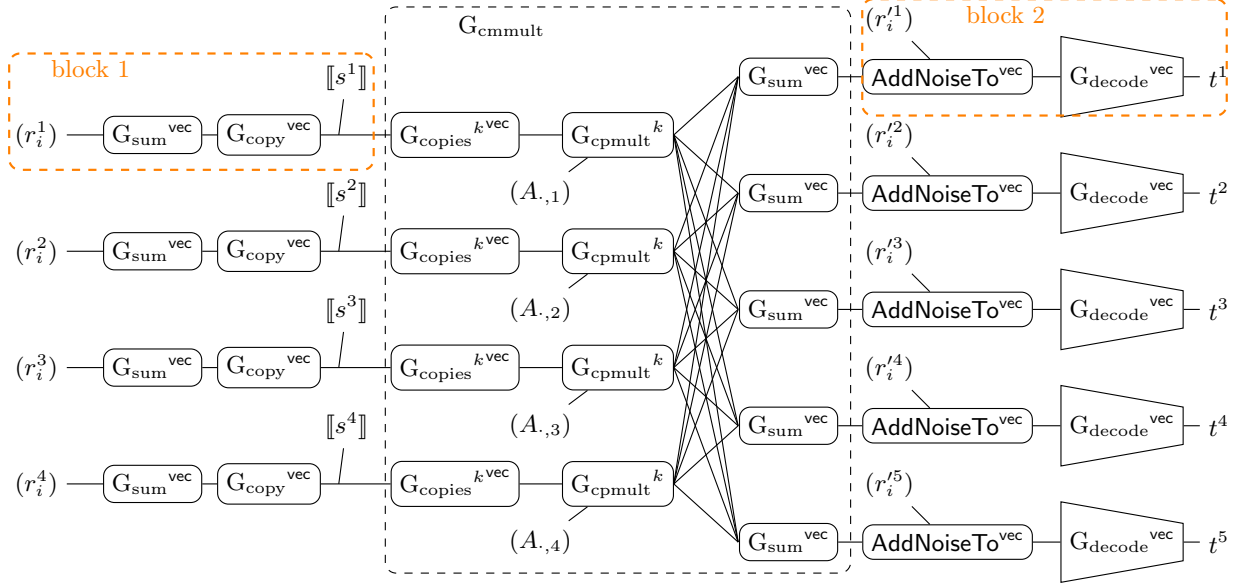


Fig. 6: RP-KeyGen for Raccoon-128 with $\ell = 4$ and $k = 5$

Theorems 3 establishes the threshold-RPC-AI-O security of Raccoon’s key generation and Raccoon’s signature processes as defined in Algorithms 13 and 15, following Definition 17. The proofs rely on the cardinal-RPC(-AI)(-O) envelopes of gadgets and their compositions, and is provided in Appendix C.

Theorem 3. *Let t' be the maximum number of leaking values of r_i and r'_i for each polynomial coefficient for both key generation and signature algorithm. The key generation and signature algorithms of Raccoon-128-16, displayed in Algorithms 13 and 15, are $(p, t = 8, t' = 15, \varepsilon)$ -threshold-RPC-AI-O⁷ with the pairs (p, ε) provided in Table 6.*

6.2 Comparison with the expansion technique and the probing secure implementation

Algorithms 13 and 15 can also be instantiated with the expanded gadgets from [BCP⁺20] (note that G_{decode} was not defined in [BCP⁺20]). Details on the expansion tools used to generate a random-probing secure Raccoon algorithm are provided in Appendix B.3. By applying new gadgets and leveraging composition proofs, we achieve significant improvements in both complexity and randomness consumption, as illustrated in Table 7.

Finally, to compare with the original probing-secure version of Raccoon, we calculate the probability of observing more than t variables in both algorithms, which gives a proven upper bound on its random probing security advantage. For $n = 16$ shares and a leakage probability of 2^{-24} , we obtain an advantage of almost 1 –meaning no proven security– for the key generation and for the signature. This highlights the benefit of our constructions to achieve random probing security compared to standard probing secure schemes.

7 Conclusion and future work

In conclusion, this work proposes new techniques towards stronger and tighter random probing security while providing all necessary components, from security definitions to concrete constructions, to show that random-probing security *is achievable* for a post-quantum scheme, namely Raccoon. We provide the first performance

⁷ Following Definition 17, we partition the auxiliary inputs into $n_r \times (\ell + k)$ vectors $\mathbf{a}_i$ containing 32 small uniform values. The threshold t' applies to each $\mathbf{a}_i$.

Table 7: Security and complexity of the key generation and the signature of Raccoon with 128 bits of (black-box) security in its original probing version and for both random probing secure scenarios with $p = 2^{-24}$.

	Key Generation			Signature		
	Original	Expansion	New Gadgets	Original	Expansion	New Gadgets
# shares	16	27	16	16	27	16
# additions	$2^{26.4}$	$2^{34.8}$	$2^{31.5}$	$2^{26.6}$	$2^{35.1}$	$2^{32.1}$
# linear mult.	$2^{26.3}$	$2^{27.1}$	$2^{26.3}$	$2^{26.6}$	$2^{27.4}$	$2^{26.6}$
# randoms	$2^{18.5}$	$2^{33.8}$	$2^{30.3}$	$2^{19.4}$	$2^{34.3}$	$2^{30.8}$
Security RPS/C	1	2^{-116}	2^{-132}	1	2^{-116}	2^{-130}

results for a random-probing secure post-quantum algorithm. We hope our design and notions will serve as building blocks for more efficient random-probing secure constructions and inspire “random-probing-friendly” designs.

In this work, we have focused on achieving random probing security for an existing version of Raccoon, namely Raccoon-128-16. It now remains to investigate variants with different parameters that would be better suited for random probing security. And, an additional relevant future work would be to refine the Small Raccoon security proof of [dKPR24] to directly combine black-box EUF-CMA security and random probing security and investigate various tradeoffs.

Another interesting avenue for future research could be to investigate if other recent techniques proposed in [DFZ19, BFO23, JMB24] could lead to different trade-offs for random probing secure versions of Raccoon. This would require extending the compilers proposed in these works to support composition with auxiliary input and/or public output, designing a secure decoding gadget within these frameworks, and comparing with our approach.

Acknowledgments. We warmly thank Katharina Boudgoust, Laurent Imbert, Loïc Masure, Laz Panard, Thomas Roche and Sara Sahraee for their meticulous and relevant feedback on this work. This work was co-funded by the European Union (ERC, AMaskZONE, 101077506) and Bpifrance (RESQUE project, France national quantum strategy). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

- ABC⁺23. Melissa Azouaoui, Olivier Bronchain, Gaëtan Cassiers, Clément Hoffmann, Yulia Kuzovkova, Joost Renes, Tobias Schneider, Markus Schönauer, François-Xavier Standaert, and Christine van Vredendaal. Protecting Dilithium against leakage revisited sensitivity analysis and improved implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):58–79, 2023.
- ADF16. Marcin Andrychowicz, Stefan Dziembowski, and Sebastian Faust. Circuit compilers with $O(1/\log(n))$ leakage rate. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EURO-CRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 586–615, Vienna, Austria, May 8–12, 2016. Springer Berlin Heidelberg, Germany.
- AIS18. Prabhajan Ananth, Yuval Ishai, and Amit Sahai. Private circuits: A modular approach. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 427–455, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- Ajt11. Miklós Ajtai. Secure computation with information leaking to an adversary. In Lance Fortnow and Salil P. Vadhan, editors, *43rd Annual ACM Symposium on Theory of Computing*, pages 715–724, San Jose, CA, USA, June 6–8, 2011. ACM Press.

- BBE⁺18. Gilles Barthe, Sonia Belaïd, Thomas Espitau, Pierre-Alain Fouque, Benjamin Grégoire, Mélissa Rossi, and Mehdi Tibouchi. Masking the GLP lattice-based signature scheme at any order. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part II*, volume 10821 of *Lecture Notes in Computer Science*, pages 354–384, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.
- BCP⁺20. Sonia Belaïd, Jean-Sébastien Coron, Emmanuel Prouff, Matthieu Rivain, and Abdul Rahman Taleb. Random probing security: Verification, composition, expansion and new constructions. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 339–368, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
- BCPZ16. Alberto Battistello, Jean-Sébastien Coron, Emmanuel Prouff, and Rina Zeitoun. Horizontal side-channel attacks and countermeasures on the ISW masking scheme. In Benedikt Gierlichs and Axel Y. Poschmann, editors, *Cryptographic Hardware and Embedded Systems – CHES 2016*, volume 9813 of *Lecture Notes in Computer Science*, pages 23–39, Santa Barbara, CA, USA, August 17–19, 2016. Springer Berlin Heidelberg, Germany.
- BCRT23. Sonia Belaïd, Gaëtan Cassiers, Matthieu Rivain, and Abdul Rahman Taleb. Unifying freedom and separation for tight probing-secure composition. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 440–472, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- BFO23. Francesco Berti, Sebastian Faust, and Maximilian Ortl. Provable secure parallel gadgets. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):420–459, 2023.
- BMRT22. Sonia Belaïd, Darius Mercadier, Matthieu Rivain, and Abdul Rahman Taleb. IronMask: Versatile verification of masking security. In *2022 IEEE Symposium on Security and Privacy*, pages 142–160, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press.
- BRT21. Sonia Belaïd, Matthieu Rivain, and Abdul Rahman Taleb. On the power of expansion: More efficient constructions in the random probing model. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 313–343, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- BRTV21. Sonia Belaïd, Matthieu Rivain, Abdul Rahman Taleb, and Damien Vergnaud. Dynamic random probing expansion with quasi linear asymptotic complexity. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part II*, volume 13091 of *Lecture Notes in Computer Science*, pages 157–188, Singapore, December 6–10, 2021. Springer, Cham, Switzerland.
- CFOS21. Gaëtan Cassiers, Sebastian Faust, Maximilian Ortl, and François-Xavier Standaert. Towards tight random probing security. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 185–214, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
- CGTZ23. Jean-Sébastien Coron, François Gérard, Matthias Trannoy, and Rina Zeitoun. Improved gadgets for the high-order masking of Dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):110–145, 2023.
- CJRR99. Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *Advances in Cryptology – CRYPTO’99*, volume 1666 of *Lecture Notes in Computer Science*, pages 398–412, Santa Barbara, CA, USA, August 15–19, 1999. Springer Berlin Heidelberg, Germany.
- DDF14. Alexandre Duc, Stefan Dziembowski, and Sebastian Faust. Unifying leakage models: From probing attacks to noisy leakage. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 423–440, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- dEK⁺23. Rafael del Pino, Thomas Espitau, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, Mélissa Rossi, and Markku-Juhani Saarinen. Raccoon. Technical report, National Institute of Standards and Technology, 2023.
- DFZ19. Stefan Dziembowski, Sebastian Faust, and Karol Zebrowski. Simple refreshing in the noisy leakage model. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part III*, volume 11923 of *Lecture Notes in Computer Science*, pages 315–344, Kobe, Japan, December 8–12, 2019. Springer, Cham, Switzerland.
- dKPR24. Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. In Leonid Reyzin and Douglas Stebila, editors, *Advances in*

- Cryptology – CRYPTO 2024, Part I*, volume 14920 of *Lecture Notes in Computer Science*, pages 409–444, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
- EEN⁺24. Muhammed F. Esgin, Thomas Espitau, Guilhem Niot, Thomas Prest, Amin Sakzad, and Ron Steinfeld. sfPlover: Masking-friendly hash-and-sign lattice signatures. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part VII*, volume 14657 of *Lecture Notes in Computer Science*, pages 316–345, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
- EFG⁺22. Thomas Espitau, Pierre-Alain Fouque, François Gérard, Mélissa Rossi, Akira Takahashi, Mehdi Tibouchi, Alexandre Wallet, and Yang Yu. Mitaka: A simpler, parallelizable, maskable variant of falcon. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 222–253, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- GJR18. Dahmun Goudarzi, Antoine Joux, and Matthieu Rivain. How to securely compute with noisy leakage in quasilinear complexity. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 547–574, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland.
- GP99. Louis Goubin and Jacques Patarin. DES and differential power analysis (the “duplication” method). In Çetin Kaya Koç and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems – CHES’99*, volume 1717 of *Lecture Notes in Computer Science*, pages 158–172, Worcester, Massachusetts, USA, August 12–13, 1999. Springer Berlin Heidelberg, Germany.
- ISW03. Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481, Santa Barbara, CA, USA, August 17–21, 2003. Springer Berlin Heidelberg, Germany.
- JMB24. Vahid Jahandideh, Bart Mennink, and Lejla Batina. An algebraic approach for evaluating random probing security with application to AES. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(4):657–689, 2024.
- LDK⁺22. Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022.
- PFH⁺22. Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON. Technical report, National Institute of Standards and Technology, 2022.
- Pre23. Thomas Prest. A key-recovery attack against mitaka in the t -probing model. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 13940 of *Lecture Notes in Computer Science*, pages 205–220, Atlanta, GA, USA, May 7–10, 2023. Springer, Cham, Switzerland.
- RP10. Matthieu Rivain and Emmanuel Prouff. Provably secure higher-order masking of AES. In Stefan Mangard and François-Xavier Standaert, editors, *Cryptographic Hardware and Embedded Systems – CHES 2010*, volume 6225 of *Lecture Notes in Computer Science*, pages 413–427, Santa Barbara, CA, USA, August 17–20, 2010. Springer Berlin Heidelberg, Germany.
- SAB⁺22. Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022.

$\mathcal{W} \leftarrow \text{LeakingWires}(G, p)$ $I_1, \dots, I_\ell \leftarrow \text{Sim}_1(G, \mathcal{W}, J_1, \dots, J_m)$ $out_1, out_2 \leftarrow \text{Sim}_2(\llbracket x_1 \rrbracket_{I_1}, \dots, \llbracket x_\ell \rrbracket_{I_\ell})$ $\text{return } (I_1, \dots, I_\ell, out_1, out_2)$	<i>Drawing the leaking wires.</i> <i>The simulator is given the indexes of the leaking outputs. It chooses indexes for the inputs.</i> <i>The simulator is given the shares at desired indexes and returns simulated values for the leaking wires (out_1) and outputs (out_2)</i>
---	---

A Security definitions

Definition 12 (Circuit Compiler from [BCP⁺20]). A circuit compiler is a triplet of algorithms (CC, Enc, Dec) defined as follows:

- CC (circuit compilation) is a deterministic algorithm that takes as input an arithmetic circuit C and outputs a randomized arithmetic circuit $\hat{C}$,
- Enc (input encoding) is a probabilistic algorithm that maps an input $\mathbf{x} \in \mathbb{K}^\ell$ to an encoded input $\hat{\mathbf{x}} \in \mathbb{K}^{\ell'}$,
- Dec (output decoding) is a deterministic algorithm that maps an encoded output $\llbracket \mathbf{y} \rrbracket \in \mathbb{K}^{m'}$ to a plain output $\mathbf{y} \in \mathbb{K}^m$,

which satisfy the following properties:

- **Correctness:** For every arithmetic circuit C of input length ℓ , and for every $\mathbf{x} \in \mathbb{K}^\ell$, we have

$$\Pr(\text{Dec}(\hat{C}(\llbracket \mathbf{x} \rrbracket)) = C(\mathbf{x}) \mid \llbracket \mathbf{x} \rrbracket \leftarrow \text{Enc}(\mathbf{x})) = 1, \text{ where } \hat{C} = \text{CC}(C).$$

- **Efficiency:** For some security parameter $\kappa \in \mathbb{N}$, the running time of $\text{CC}(C)$ is $\text{poly}(\kappa, |C|)$, the running time of $\text{Enc}(\mathbf{x})$ is $\text{poly}(\kappa, |\mathbf{x}|)$ and the running time of $\text{Dec}(\llbracket \mathbf{y} \rrbracket)$ is $\text{poly}(\kappa, |\llbracket \mathbf{y} \rrbracket|)$, where $\text{poly}(\kappa, \ell) = \mathcal{O}(\kappa^{e_1} \ell^{e_2})$ for some constants e_1, e_2 .

Definition 13 ((Threshold) Random Probing Composability from [BCP⁺20]). Let $n, \ell, m \in \mathbb{N}$. An n -share gadget $G : (\mathbb{K}^n)^\ell \rightarrow (\mathbb{K}^n)^m$ is (t, p, ϵ) -random probing composable (RPC) for some $t \in \mathbb{N}$ and $p, \epsilon \in [0, 1]$ if there exists a PPT stateful two-stage simulator $(\text{Sim}_1, \text{Sim}_2)$ such that for every masked input $\llbracket \mathbf{x} \rrbracket \in (\mathbb{K}^n)^\ell$ and for every set collection $J_1 \subseteq [n], \dots, J_m \subseteq [n]$ of cardinals $|J_1| \leq t, \dots, |J_m| \leq t$, the outputs of Experiment 5 are such that

1. $\Pr((|I_1| > t) \vee \dots \vee (|I_\ell| > t)) \leq \epsilon$
- 2.

$$(out_1, out_2) \stackrel{id}{=} \left(\underbrace{\mathcal{L}_{\mathcal{W}}(G, \llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{y} \rrbracket)}_{\text{Induced Random probing leakage}}, \underbrace{(\llbracket y_1 \rrbracket_{J_1}, \dots, \llbracket y_m \rrbracket_{J_m})}_{\text{Output leakage}} \right)$$

where $\mathcal{W} \leftarrow \text{LeakingWires}(C, p)$ and $\llbracket \mathbf{y} \rrbracket \leftarrow G(\llbracket \mathbf{x} \rrbracket)$.

Definition 14. Let $f : \mathbb{R} \rightarrow \mathbb{R}$. The gadget G is (t, f) -RPC if it is $(t, p, f(p))$ -RPC for every $p \in [0, 1]$.

Theorem 4 (Composition from [BCP⁺20]). Let $t \in \mathbb{N}$, $p, \epsilon \in [0, 1]$, and CC be a standard circuit compiler with (t, p, ϵ) -RPC base gadgets. For every (randomized) arithmetic circuit C composed of $|C|$ gadgets, the compiled circuit $\text{CC}(C)$ is $(p, |C| \cdot \epsilon)$ -random probing secure. Equivalently, the standard circuit compiler CC is (p, ϵ) -random probing secure.

$\mathcal{W} \leftarrow \text{LeakingWires}(G, p)$ $I_1, \dots, I_\ell \leftarrow \text{Sim}_1(G, \mathcal{W}, J_1, \dots, J_m)$ $out_1, out_2 \leftarrow \text{Sim}_2(\llbracket x_1 \rrbracket_{I_1}, \dots, \llbracket x_\ell \rrbracket_{I_\ell})$ $\text{return } (I_1, \dots, I_\ell, out_1, out_2)$	<i>Drawing the leaking wires.</i> <i>The simulator is given the indexes of the leaking outputs. It chooses indexes for the inputs.</i> <i>The simulator is given the shares at desired indexes and returns simulated values for the leaking wires (out_1) and outputs (out_2)</i>
---	---

Definition 15 (General Random Probing Composability from [CFOS21]). Let $n, \ell, m \in \mathbb{N}$. Let $\mathcal{E}$ represent a collection of probability envelopes indexed by $(J_1, \dots, J_m) \in \mathbb{K}^{|J_1|} \times \dots \times \mathbb{K}^{|J_m|}$, where $|J_i| \in [0, n]$ for every $i \in [1, m]$. These envelopes are defined over tuples of size ℓ of elements in $\mathbb{K}^i$ for $i \in [0, n]$. An n -share gadget $G : (\mathbb{K}^n)^\ell \rightarrow (\mathbb{K}^n)^m$ is $(p, \mathcal{E})$ -general random probing composable (general-RPC for short) for some $p \in [0, 1]$ if there exists a PPT stateful two-stage simulator $(\text{Sim}_1, \text{Sim}_2)$ such that for every shared input $\llbracket x \rrbracket \in (\mathbb{K}^n)^\ell$ and for every set collection $(J_1, \dots, J_m)$, the outputs of Experiment 6 are such that

1. $(I_1, \dots, I_\ell) \lesssim \mathcal{E}_{J_1, \dots, J_m}$,
- 2.

$$(out_1, out_2) \stackrel{id}{=} \left(\underbrace{\mathcal{L}_{\mathcal{W}}(G, \llbracket x \rrbracket, \llbracket y \rrbracket)}_{\text{Induced Random probing leakage}}, \underbrace{(\llbracket y_1 \rrbracket_{J_1}, \dots, \llbracket y_m \rrbracket_{J_m})}_{\text{Output leakage}} \right),$$

where $\mathcal{W} \leftarrow \text{LeakingWires}(C, p)$ and $\llbracket y \rrbracket \leftarrow G(\llbracket x \rrbracket)$.

Definition 16 (Uniformly Cardinal Random Probing Composability). Let $n, \ell, m \in \mathbb{N}$. Let $\mathcal{E}^u$ represent a collection of probability envelopes indexed by $(j_1, \dots, j_m) \subseteq [0, n]^m$ over $[0, n]^\ell$. An n -share gadget $G : (\mathbb{K}^n)^\ell \rightarrow (\mathbb{K}^n)^m$ is uniformly $(p, \mathcal{E}^u)$ -cardinal random probing composable (uniformly cardinal-RPC for short) for some $p \in [0, 1]$ if it is $(p, \mathcal{E}^g)$ -general-RPC and if for all $(J_1, \dots, J_m), (J'_1, \dots, J'_m) \in \mathbb{K}^{|J_1|} \times \dots \times \mathbb{K}^{|J_m|}$ such that

$$(|J_1|, \dots, |J_m|) = (|J'_1|, \dots, |J'_m|)$$

and for all $(I_1, \dots, I_\ell), (I'_1, \dots, I'_\ell) \in \mathbb{K}^{|I_1|} \times \dots \times \mathbb{K}^{|I_\ell|}$ such that

$$(|I_1|, \dots, |I_\ell|) = (|I'_1|, \dots, |I'_\ell|),$$

$$\text{we have } \mathcal{E}_{J_1, \dots, J_m}^g(I_1, \dots, I_\ell) = \mathcal{E}_{J'_1, \dots, J'_m}^g(I'_1, \dots, I'_\ell).$$

The collection of uniformly cardinal-RPC envelopes $\mathcal{E}^u$ is then naturally defined such that

$$\mathcal{E}_{|J_1|, \dots, |J_m|}^u(|I_1|, \dots, |I_\ell|) = \mathcal{E}_{J_1, \dots, J_m}^g(I_1, \dots, I_\ell).$$

Definition 17 ((Threshold) Random Probing Composability with Auxiliary Inputs and Public Outputs).

Let $n, \ell, m, k, \alpha, d \in \mathbb{N}$. Let G be a gadget with the following input/output partition

$$G : \overbrace{(\mathbb{K}^n)^\ell}^{\text{masked inputs}} \times \overbrace{(\mathbb{K}^\alpha)^k}^{\text{auxiliary inputs}} \rightarrow \overbrace{(\mathbb{K}^n)^m}^{\text{masked outputs}} \times \overbrace{\mathbb{K}^d}^{\text{public outputs}}$$

$$(\llbracket x \rrbracket, \mathbf{a}_1, \dots, \mathbf{a}_k) \mapsto (\llbracket y \rrbracket, \mathbf{z}).$$

The gadget G is (p, t, t', ε) -threshold random probing composable with Auxiliary Inputs and Public Outputs for some $p \in [0, 1]$, $t, t' \in \mathbb{N}$ and $\varepsilon > 0$ if there exists a PPT stateful two-stage simulator $(\text{Sim}_1, \text{Sim}_2)$ such

$(*, \mathbf{z}) \leftarrow C(\text{Enc}(\mathbf{x}), \mathbf{a})$ $\mathcal{W} \leftarrow \text{LeakingWires}(C, p)$ $(I_1, \dots, I_\ell), (L_1, \dots, L_k) \leftarrow \text{Sim}_1(C, \mathcal{W}, \mathbf{z}, J_1, \dots, J_m)$	<i>Drawing the public outputs.</i> <i>Drawing the leaking wires.</i> <i>The simulator is given</i> <i>the indexes of the leaking</i> <i>outputs. It chooses</i> <i>indexes for the shared</i> <i>and auxiliary inputs.</i>
$out_1, out_2 \leftarrow \text{Sim}_2(\llbracket x_1 \rrbracket_{I_1}, \dots, \llbracket x_\ell \rrbracket_{I_\ell}, \mathbf{a}_1 _{L_1}, \dots, \mathbf{a}_k _{L_k})$	<i>The simulator is given the</i> <i>shares at desired indexes</i> <i>and returns simulated</i> <i>values for the leaking</i> <i>wires and outputs.</i>
$\text{return } (out_1, out_2, (I_1, \dots, I_\ell), (L_1, \dots, L_k))$	

that for every shared input $\llbracket x \rrbracket \in (\mathbb{K}^n)^\ell$, auxiliary input $\mathbf{a}_1, \dots, \mathbf{a}_k \in (\mathbb{K}^\alpha)^k$, and for every set collection $(J_1, \dots, J_m)$ where $J_1 \subseteq [n], \dots, J_m \subseteq [n]$ of cardinals $|J_1| \leq t, \dots, |J_m| \leq t$, the outputs of Experiment 7 are such that

1. $\Pr(|I_1| > t) \vee \dots \vee (|I_\ell| > t) \leq \varepsilon$
2. $\Pr(|L_1| > t') \vee \dots \vee (|L_k| > t') \leq \varepsilon$
3. and

$$(out_1, out_2) \stackrel{id}{=} \left(\underbrace{\mathcal{L}_{\mathcal{W}}(G, (\llbracket \mathbf{x} \rrbracket, \mathbf{a}), (\llbracket \mathbf{y} \rrbracket, \mathbf{z}))}_{\text{Induced Random probing leakage}}, \underbrace{(\llbracket y_1 \rrbracket_{J_1}, \dots, \llbracket y_m \rrbracket_{J_m})}_{\text{Output leakage}} \right)$$

where $\mathcal{W} \leftarrow \text{LeakingWires}(C, p)$ and $\llbracket \mathbf{y} \rrbracket, \mathbf{z} \leftarrow G(\llbracket \mathbf{x} \rrbracket, \mathbf{a})$.

B Expansion Strategy

B.1 Random Probing Expandability

In [AIS18], an *expansion* approach was introduced to build a random-probing secure circuit compiler from a secure multi-party protocol. This approach was later revisited in [BCP⁺20] with the formalization of the notion of *expanding compiler* [BCP⁺20].

The principle of the expanding compiler is to recursively apply a base compiler, denoted CC and which simply consists in replacing each gate in the input circuit by the corresponding gadget. Assume we have n -share gadgets G_g for each gate g in a basis $\mathbb{B}$. The base compiler CC simply consists in replacing each gate g in these gadgets by G_g and by replacing each wire by n wires carrying a sharing of the value. We thus obtain n^2 -share gadgets by simply applying CC to each gadget: $G_g^{(2)} = \text{CC}(G_g)$. This process can be iterated an arbitrary number of times, say k , to an input circuit C :

$$C \xrightarrow{\text{CC}} \widehat{C}_1 \xrightarrow{\text{CC}} \dots \xrightarrow{\text{CC}} \widehat{C}_k.$$

The first output circuit $\widehat{C}_1$ is the original circuit in which each gate g is replaced by a base gadget G_g . The second output circuit $\widehat{C}_2$ is the original circuit C in which each gate is replaced by an n^2 -share gadget $G_g^{(2)}$. Equivalently, $\widehat{C}_2$ is the circuit $\widehat{C}_1$ in which each gate is replaced by a base gadget. In the end, the output

Experiment 8: RPE

$\mathcal{W} \leftarrow \text{LeakingWires}(G, p)$ $(I_1, I_2, J') \leftarrow \text{Sim}_1(G, \mathcal{W}, J)$ $out_1, out_2 \leftarrow \text{Sim}_2(\llbracket x_1 \rrbracket _{I_1}, \llbracket x_2 \rrbracket _{I_2})$ $\text{return } (I_1, I_2, J', out_1, out_2).$	<i>Drawing the leaking wires.</i> <i>The simulator is given the indexes of the leaking outputs (J). It chooses indexes for the inputs (I_1, I_2).</i> <i>The simulator is given the shares at desired indexes and returns simulated values for the leaking wires (out_1) and outputs (out_2).</i>
---	---

circuit $\hat{C}_k$ is hence the original circuit C in which each gate has been replaced by a k -expanded gadget and each wire has been replaced by n^k wires carrying an (n^k) -linear sharing of the original wire.

The expanding compiler achieves random probing security if the base gadgets verify a property called *random probing expandability* [BCP⁺20]. We recall hereafter the original definition of the random probing expandability (RPE) property for 2-to-1 gadgets.

Definition 18 (Random Probing Expandability from [BCP⁺20]). *Let $f : \mathbb{R} \rightarrow \mathbb{R}$. An n -share 2-to-1 gadget $G : \mathbb{K}^n \times \mathbb{K}^n \rightarrow \mathbb{K}^n$ is (t, f) -random probing expandable ((t, f) -RPE for short) if there exists a PPT stateful two-stage simulator $(\text{Sim}_1, \text{Sim}_2)$ such that for every admissible pair $((\llbracket x_1 \rrbracket, \llbracket x_2 \rrbracket), \llbracket y \rrbracket) \in (\mathbb{K}^n \times \mathbb{K}^n) \times \mathbb{K}^n$, for every set $J \subseteq [n]$ and for every $p \in [0, 1]$, the outputs of Experiment 8 are such that*

1. $\Pr(|I_1| > t) = \Pr(|I_2| > t) = f(p)$ and $\Pr((|I_1| > t) \wedge (|I_2| > t)) = f(p)^2$,
2. $J' \subseteq [n]$ is such that $J' = J$ if $|J| \leq t$ and $|J'| = n - 1$ otherwise,
3. and

$$(out_1, out_2) \stackrel{id}{=} \left(\underbrace{\mathcal{L}_{\mathcal{W}}(G, (\llbracket x_1 \rrbracket, \llbracket x_2 \rrbracket), \llbracket y \rrbracket))}_{\text{Induced Random probing leakage}}, \underbrace{\llbracket y \rrbracket|_{J'}}_{\text{Output leakage}} \right)$$

where $\mathcal{W} \leftarrow \text{LeakingWires}(C, p)$.

The first condition of Definition 18 defines two events, called "failure events" $\mathcal{F}_1 \equiv (|I_1| > t)$ and $\mathcal{F}_2 \equiv (|I_2| > t)$. In particular, condition 1 implies that $\mathcal{F}_1$ and $\mathcal{F}_2$ are mutually independent. The second condition may seem artificial as the simulator is able to modify the indexes of leaking outputs. This is actually necessary to capture some specific events (when $|J| \geq t$) in the proofs, we refer to [BCP⁺20] for more details.

Remark 2. The RPE notion can be simply extended to gadgets with 2 outputs: the Sim_1 simulator takes two sets $J_1 \subseteq [n]$ and $J_2 \subseteq [n]$ as input and produces two sets J'_1 and J'_2 satisfying the same property as J' in the above definition (w.r.t. J_1 and J_2). The Sim_2 simulator must then produce an output with the knowledge of $\llbracket y_1 \rrbracket|_{J'_1}$ and $\llbracket y_2 \rrbracket|_{J'_2}$ where $\llbracket y_1 \rrbracket$ and $\llbracket y_2 \rrbracket$ are the output sharings.

The RPE notion can also be simply extended to gadgets with a single input: the Sim_1 simulator produces a single set I so that the failure event $(|I| > t)$ occurs with probability ε (and the Sim_2 simulator is then simply given $\llbracket x \rrbracket|_I$ where $\llbracket x \rrbracket$ is the single input sharing). We refer the reader to [BCP⁺20] for the formal definitions of these variants.

B.2 Instantiation of Noise Generation from RPE

In this section, we rely on the expansion strategy from [BCP⁺20] (and recalled in Section B.1) to build a noise generation gadget from an addition gadget. Specifically, based on the analysis and instantiations

from [BRT21], we begin with the following small addition and copy gadgets of 3 and 5 shares, which we then expand to increase their original security level for various leakage probabilities:

$$\begin{array}{ll}
\mathbf{G}_{\text{add}}^3 : (\mathbb{K}^3)^2 \rightarrow \mathbb{K}^3 & \mathbf{G}_{\text{copy}}^3 : \mathbb{K}^3 \rightarrow (\mathbb{K}^3)^2 \\
((a_1, a_2, a_3), (b_1, b_2, b_3)) \mapsto (c_1, c_2, c_3) & (a_1, a_2, a_3) \mapsto ((c_1, c_2, c_3), (d_1, d_2, d_3)) \\
\\
\begin{array}{l}
c_1 \leftarrow (r_1 + a_1) + (r_3 + b_1) \\
c_2 \leftarrow (r_2 + a_2) + (r_4 + b_2) \\
c_3 \leftarrow (r_1 + r_2 + a_3) + (r_3 + r_4 + b_3)
\end{array} & \begin{array}{ll}
c_1 \leftarrow r_1 + a_1 & d_1 \leftarrow r_3 + a_1 \\
c_2 \leftarrow r_2 + a_2 & d_2 \leftarrow r_4 + a_2 \\
c_3 \leftarrow r_1 + r_2 + a_3 & d_3 \leftarrow r_3 + r_4 + a_3
\end{array}
\end{array}$$

and

$$\begin{array}{ll}
\mathbf{G}_{\text{add}}^5 : (\mathbb{K}^5)^2 \rightarrow \mathbb{K}^5 & \mathbf{G}_{\text{copy}}^5 : (\mathbb{K}^5)^2 \rightarrow \mathbb{K}^5 \\
((a_i)_{1 \leq i \leq 5}, (b_i)_{1 \leq i \leq 5}) \mapsto (c_i)_{1 \leq i \leq 5} & (a_i)_{1 \leq i \leq 5} \mapsto ((c_i)_{1 \leq i \leq 5}, (d_i)_{1 \leq i \leq 5}) \\
\\
\begin{array}{l}
c_1 \leftarrow (r_1 + r_2 + a_1) + (r_6 + r_7 + b_1) \\
c_2 \leftarrow (r_2 + r_3 + a_2) + (r_7 + r_8 + b_2) \\
c_3 \leftarrow (r_3 + r_4 + a_3) + (r_8 + r_9 + b_3) \\
c_4 \leftarrow (r_4 + r_5 + a_4) + (r_9 + r_{10} + b_4) \\
c_5 \leftarrow (r_5 + r_1 + a_5) + (r_{10} + r_6 + b_5)
\end{array} & \begin{array}{ll}
c_1 \leftarrow r_1 + r_2 + a_1 & d_1 \leftarrow r_6 + r_7 + a_1 \\
c_2 \leftarrow r_2 + r_3 + a_2 & d_2 \leftarrow r_7 + r_8 + a_2 \\
c_3 \leftarrow r_3 + r_4 + a_3 & d_3 \leftarrow r_8 + r_9 + a_3 \\
c_4 \leftarrow r_4 + r_5 + a_4 & d_4 \leftarrow r_9 + r_{10} + a_4 \\
c_5 \leftarrow r_5 + r_1 + a_5 & d_5 \leftarrow r_{10} + r_6 + a_5
\end{array}
\end{array}$$

Using **IronMask** from [BMRT22], we obtain the RPE advantage of both gadgets for $t = 1$ with 3 shares and $t = 2$ with 5 shares and for any probability p . For 3 shares, we have:

$$f_3(p) \approx 171p^2 + 36.4p^2\sqrt{p} + 72p^3 + 364.9p^3\sqrt{p} + 12648p^4 + \mathcal{O}(p^5)$$

with a maximum leakage probability (*i.e.* the maximum p for which $f(p) < p$) of $2^{-7.4}$, and for 5 shares, we have:

$$f_5(p) \approx 2870p^3 + 476.4p^3\sqrt{p} + 75p^4 + 9230.2p^5 + \mathcal{O}(p^6)$$

with a maximum leakage probability (*i.e.* the maximum p for which $f(p) < p$) of $2^{-5.77}$. Table 8 displays the levels of expansion k that are required for gadgets $\mathbf{G}_{\text{add}}^3$ and $\mathbf{G}_{\text{add}}^5$ and different probabilities to reach 128 bits of security, with the corresponding complexities. The latter are computed using the expanding compiler from [BCP⁺20].

As expected, the resulting complexities for the same leakage probability and security threshold are better for $\mathbf{G}_{\text{add}}^5$. While we could theoretically further increase the number of shares before the expansion, we are limited by the complexity of the automatic tools (here **IronMask**).

Table 8: Security and complexity of expansions of gadgets G_{add}^3 and G_{add}^5 .

	Gadget G_{add}^3					Gadget G_{add}^5				
leakage prob. p	2^{-8}	2^{-12}	2^{-16}	2^{-20}	2^{-24}	2^{-8}	2^{-12}	2^{-16}	2^{-20}	2^{-24}
expansion level k	8	5	4	4	3	4	3	3	2	2
# shares n	6561	243	81	81	27	625	125	125	25	25
# leaking shares t	2187	81	27	27	9	250	50	50	10	10
# randoms	854M	253k	17k	17k	1k	500k	14k	14k	400	400
# additions	1709M	506k	34k	34k	2k	1001k	29k	29k	825	825
# copies	854M	253k	17k	17k	1k	500k	14k	14k	400	400
advantage $f^k(p)$	2^{-152}	2^{-154}	2^{-145}	2^{-209}	2^{-140}	2^{-188}	2^{-175}	2^{-283}	2^{-134}	2^{-170}

Using the expansion method, the resulting G_{add} gadgets are only RPC for a fixed t once the level of expansion is determined (according to the probability and the target security level). Therefore, we rely on the composition theorem from [BCP⁺20] to compute the RPC security of **AddNoiseTo** for selected probabilities and size α of the auxiliary input. Specifically, the security advantage of **AddNoiseTo** is upper bounded by the product of the security advantage of G_{add} with the number of such gadgets. The whole gadget requires t (with its post-expansion value) regular input shares (of $\llbracket x \rrbracket$) and $t \cdot \lceil \frac{\alpha}{n} \rceil$ auxiliary inputs for the simulation of the leakage and t output shares. For instance, with $\alpha = 75$ auxiliary inputs and a leakage probability $p = 2^{-24}$, we need to combine 3 gadgets G_{add} to form a gadget **AddNoiseTo**. The resulting RPC advantage is upper bounded by 2^{-169} , and 10 shares from the main input and 30 auxiliary inputs are required for the simulation. As for the complexity, the noise gadget requires 1200 randoms and implements 2475 additions and 1200 copies. In this setting, the selection of $G_{\text{sum-chain}}$ or $G_{\text{sum-tree}}$ does not affect neither the security, nor the complexity (except when parallelization is activated).

B.3 Instantiation of Raccoon from RPE

To implement the key generation and the signature with the expansion strategy, we miss a linear multiplication gadget, G_{cmult} , for which we give instantiations below for 3 and 5 shares. We simply follow the structures of G_{add} and G_{copy} with a refresh of the inputs preceding the linear multiplication.

To implement key generation and signature using the expansion strategy, we need a linear multiplication gadget, G_{cmult} . Below, we provide specific instantiations for 3 and 5 shares. The design mirrors the structures of G_{add} and G_{copy} , with the inputs refreshed before performing the linear multiplication.

$$\begin{aligned}
 G_{\text{cmult}}^3 : \mathbb{K}^3 \times \mathbb{K} &\rightarrow \mathbb{K}^3 \\
 ((a_1, a_2, a_3), \alpha) &\mapsto (c_1, c_2, c_3) \\
 c_1 &\leftarrow \alpha \cdot (r_1 + a_1) \\
 c_2 &\leftarrow \alpha \cdot (r_2 + a_2) \\
 c_3 &\leftarrow \alpha \cdot (r_1 + r_2 + a_3)
 \end{aligned}
 \qquad
 \begin{aligned}
 G_{\text{cmult}}^5 : \mathbb{K}^5 \times \mathbb{K} &\rightarrow \mathbb{K}^5 \\
 ((a_1, a_2, a_3, a_4, a_5), \alpha) &\mapsto (c_1, c_2, c_3, c_4, c_5) \\
 c_1 &\leftarrow \alpha \cdot (r_1 + r_2 + a_1) \\
 c_2 &\leftarrow \alpha \cdot (r_2 + r_3 + a_2) \\
 c_3 &\leftarrow \alpha \cdot (r_3 + r_4 + a_3) \\
 c_4 &\leftarrow \alpha \cdot (r_4 + r_5 + a_4) \\
 c_5 &\leftarrow \alpha \cdot (r_5 + r_1 + a_5)
 \end{aligned}$$

We use **IronMask** from [BMRT22] to compute the new RPE advantages for G_{cmult} combined with G_{add} and G_{copy} . They are identical to those obtained for G_{add} without G_{cmult} in Section B.2.

In Table 8, we identify a single configuration where the number of shares n and the number of leaking shares t correspond to an existing version in Raccoon (version 16), specifically considering the total number

of small uniforms r_i and r'_i and the number of them that can leak. We denote by m_r the number of small uniforms that are added to each coefficient in $\mathbb{Z}_q$ and by m_s the number of small uniforms that cannot be revealed for each coefficient in $\mathbb{Z}_q$. Using our expansion strategy, the small uniforms must be captured inside a n -share variable, that is, there exists $\alpha \in \mathbb{N}^*$ such that

$$(\alpha - 1)n < m_r \leq \alpha n.$$

Then, we must ensure that the number of input shares that are required for the leakage simulation is lower than $m_r - m_s$. This constraint can be written as follows:

$$(\alpha - 1)t + \min(t, n - (\alpha n - m_r)) \leq m_r - m_s.$$

It can be observed that the only column in Table 8) satisfying this inequality corresponds to the use of gadget G_{add}^3 with $n = 27$ shares and an expansion level $k = 3$. For each coefficient in $\mathbb{Z}_q$, 27 small uniforms can be packed into a first sharing, while the remaining 5 small uniforms are packed into a second sharing, supplemented with 22 zeros. The RPC advantage of Raccoon's construction under this configuration then represents the probability that $t + 5 = 14$ small uniforms are required to simulate the leakage and t outputs, with $32 - 14 = 18$ small uniforms remaining secret, exceeding the expected 17 ones.

With these parameters in Raccoon-128, we count almost 16 million base gadgets for key generation and nearly 19 million base gadgets for the signature. Since no implementation is available for the decoding gadget using the expansion strategy, its impact on complexity and security is omitted. Consequently, the security advantage is upper-bounded by 2^{-116} in both cases, for a leakage probability of 2^{-24} . This is calculated by multiplying the atomic security advantage of a single gadget (*i.e.* 2^{-140} from Table 8) by the total number of gadgets. The complexity with this expansion strategy, including the number of logical gates and intermediate random values, is provided in Table 7 and can be compared to the original scheme with 16 shares.

C Proofs of Raccoon's Random Probing Security

C.1 Security Proof for Raccoon's Key Generation

Proof (Proof of Theorem 3). Due to the complexity of the key generation circuit, calculating a global cardinal-RPC-AI envelope is computationally impractical. Even before considering the internal wires, we already have $(\ell + k)n_R = 4.608$ n -share inputs and $\ell n_R = 2.048$ outputs, for which enumerating all possible values in $[0, n]$ would be too expensive. As a result, we opt to decompose the scheme into sub-components, each of which is individually proven to be cardinal-RPC-AI-O. For these sub-components, we calculate the corresponding threshold-RPC-AI-O, using a fixed threshold $t = \frac{n}{2}$. Finally, we sum the threshold-RPC-AI-O advantages of these blocks to determine the overall threshold-RPC-AI-O security of the complete scheme.

Specifically, we identify three different types of blocks:

- Block 1, illustrated on Figure 6, consists of G_{sum} and G_{copy} gadgets. We first compute the cardinal-RPC-AI envelope of the combination of both gadgets G_{sum} and G_{copy} , then the threshold-RPC-AI advantage for an attacker who gets $t = \frac{n}{2}$ shares of each output and when the auxiliary input r_i^j is constrained to have a cardinality of at most $t' = 15$ (per polynomial coefficient). The resulting threshold-RPC-AI advantage is then multiplied by $n_R = 512$ and $\ell = 4$ (following an extension of the composition property of [BCP⁺20]) to obtain an upper bound on the threshold-RPC-AI advantage for the left part of the circuit, whose outputs will be used as inputs for G_{cmmult} .
- Gadget G_{cmmult} is first split into three blocks. The ℓn_R $\mathbb{Z}_q$ n -share outputs of the left block are first each copied k times. We compute the cardinal-RPC envelope of the corresponding G_{copies}^k gadget from which we derive its threshold-RPC advantage with t output shares and when we restrict the cardinality of the input shares to t as well. Then each of the ℓk copies in $\mathcal{R}_q$ goes through a linear polynomial multiplication with gadget G_{cpmult} . Gadget G_{cpmult} consists in the generation of n_R copies of n_R elements in $\mathbb{Z}_q$. We are able to tightly compute the threshold-RPC advantage of the combination of G_{copy} gadgets

assembled in a tree to compute n_R copies based on the cardinal-RPC envelopes of each block and by restricting the cardinal of the output shares to t and the cardinal of the input shares to at most t . Then, n_R^2 linear multiplications in $\mathbb{Z}_q$ are performed for which we have the cardinal-RPC envelope from Section 4.2 and from which we can derive the threshold-RPC advantage when restricting the input and output cardinal with respect to t . Finally, we have n_R groups of n_R elements of $\mathbb{Z}_q$ that are summed together. We use gadget G_{sum} with the tree structure to compute additions of 4 shared variables (we need $128 + 32 + 8 + 4 + 1 = 173$ of them). From the cardinal-RPC envelopes of G_{sum} , we derive their threshold-RPC advantage with t and we sum them to obtain the global threshold-RPC advantage of all the additions. Summing all the computed threshold-RPC advantages (following the composition property of [BCP⁺20]) gives us an upper bound the threshold-RPC advantage of G_{cmmult} .

- Lastly, we compute the global threshold-RPC-AI-O advantage for the right block, whose inputs are the outputs of G_{cmmult} . We begin by determining the cardinal-RPC-AI-O envelope for the combination of **AddNoiseTo** and G_{decode} , then we compute the threshold-RPC-AI-O advantage when we restrict the input shared cardinality to be less or equal to t and the cardinality of the auxiliary inputs $r_i^{j'}$ to be at most 15. Similar to the process in block 1, we then multiply the resulting threshold-RPC-AI advantage by $n_R = 512$ and $k = 5$ (in line with the composition property from [BCP⁺20]), yielding an upper bound on the threshold-RPC-AI-O advantage for this right block.

All these advantages are computed for different probabilities and for instantiations of RPRrefresh with different values of γ for each of the three blocks. The code is provided in Supplementary Material. The resulting threshold-RPC-AI-O advantages are displayed with the corresponding complexities in Table 7. $\square$

C.2 Security Proof for Raccoon’s Signature

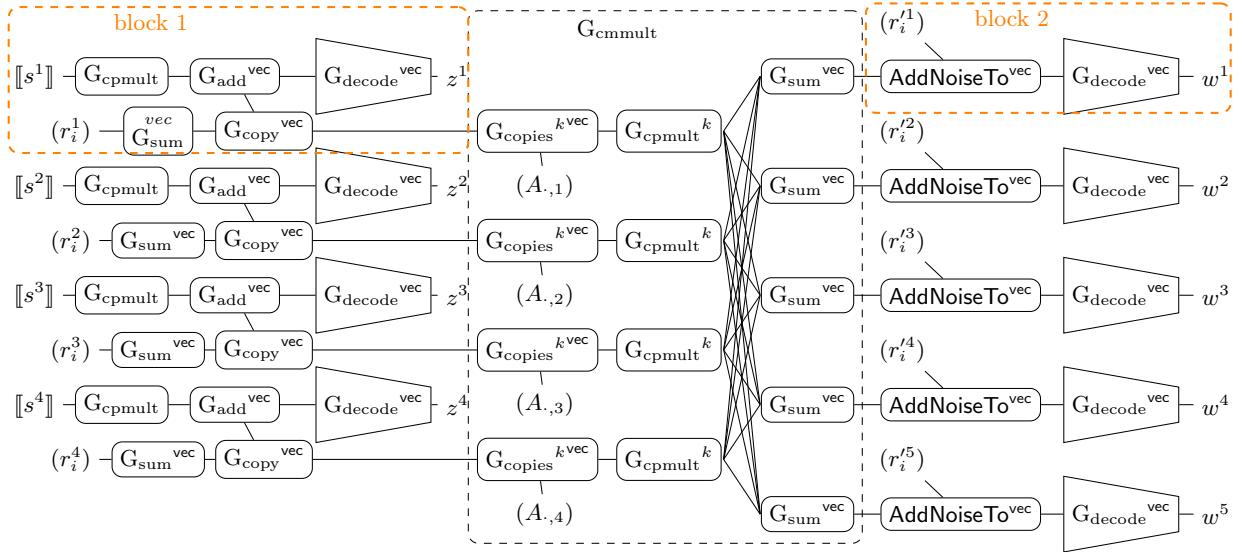


Fig. 7: RP-Sign for Raccoon-128 with $\ell = 4$ and $k = 5$

Proof. The proof for Raccoon’s signature is very close to the proof for Raccoon’s key generation, except for the computation of the threshold-RPC-AI-O advantage of the first block. Specifically, we divide this block into three sub-blocks. The first sub-block contains G_{sum} and G_{copy} gadgets. It is identical to block 1 of the key generation. We reuse the threshold-RPC-AI-O advantage for t as computed for the key generation. The

second sub-block contains G_{decode} and G_{add} gadgets, we compose their envelopes and derive the corresponding threshold-RPC-AI-O advantage for t . Finally, we compute the threshold-RPC advantage of G_{cpmult} for t , as in the proof of Theorem 3. Following [BCP⁺20], we sum the advantages of all three parts (the first ones being multiplied by ℓn_R and the last advantage of G_{cpmult} being multiplied by ℓ) to obtain the global threshold-RPC-AI-O of block 1. The script to compute the global threshold-RPC-AI-O of Raccoon's signature is provided in Supplementary Material and a few values are provided in Table 7. $\square$